

A Major Project Report  
On  
**GRAPH BASED CONVOLUTIONAL NETWORK BASED MODEL  
FOR MEGACITY REAL ESTATE VALUATION**

Submitted to CMREC (UGC Autonomous)

In Partial Fulfilment of the requirements for the Award of Degree  
of  
**BACHELOR OF TECHNOLOGY**  
IN  
**COMPUTER SCIENCE AND ENGINEERING (AI & ML)**

Submitted  
By

|                           |                     |
|---------------------------|---------------------|
| <b>SOMA JHANSI</b>        | <b>(228R1A66C2)</b> |
| <b>GUNDA MAHITHA</b>      | <b>(228R1A6687)</b> |
| <b>AJMEERA SHRUTI</b>     | <b>(228R1A6668)</b> |
| <b>KANKANALA VASUDEVA</b> | <b>(228R1A6698)</b> |

Under the Esteemed guidance of

**Mrs. M. SRIKALA**

Assistant Professor, Department of CSE(AI&ML)



**Department of Computer Science and Engineering(AI&ML)**

**CMR ENGINEERING COLLEGE**  
**(UGC AUTONOMOUS)**

(Accredited by NAAC & NBA, Approved by AICTE, New Delhi, Affiliated to JNTU, Hyderabad)  
(Kandlakoya, Medchal Road, Medchal-Malkajgiri Dist., Hyderabad-501 401)

**(2025-2026)**

# CMR ENGINEERING COLLEGE

## UGC AUTONOMOUS

*(Accredited by NAAC&NBA, Approved by AICTE New Delhi, Affiliated to JNTU,*

*Hyderabad, Kandlakoya, Medchal Road, Hyderabad-501 401)*

## Department of Computer Science & Engineering (AI & ML)



### CERTIFICATE

This is to certify that the Major project entitled “**GRAPH BASED CONVOLUTIONAL NETWORK BASED MODEL FOR MEGACITY REAL ESTATE VALUATION** ” is a bonafide work carried out by

|                           |                     |
|---------------------------|---------------------|
| <b>SOMA JHANSI</b>        | <b>(228R1A66C2)</b> |
| <b>GUNDA MAHITHA</b>      | <b>(228R1A6687)</b> |
| <b>AJMEERA SHRUTI</b>     | <b>(228R1A6668)</b> |
| <b>KANKANALA VASUDEVA</b> | <b>(228R1A6698)</b> |

in partial fulfillment of the requirement for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING (AI&ML) from CMR Engineering College, under our guidance and supervision.

The results presented in this Major project have been verified and are found to be satisfactory. The results embodied in this Major project have not been submitted to any other university for the award of any other degree or diploma.

---

**Internal Guide**

Mrs.M.Srikala  
Assistant Professor  
Department of  
CSE (AI & ML)

---

**Major Project Coordinator**

Mr. G. Venkateswarlu  
Assistant Professor  
Department of  
CSE (AI & ML)

---

**Head of the Department**

Dr. Madhavi Pingili  
Professor & HOD  
Department of  
CSE (AI & ML)

**External Examiner:** \_\_\_\_\_

## **DECLARATION**

This is to certify that the work reported in the present Major project entitled “**GRAPH BASED CONVOLUTIONAL NETWORK BASED MODEL FOR MEGACITY REAL ESTATE VALUATION** ” is a record of bonafide work done by us in the Department of Computer Science and Engineering (AI & ML), CMR Engineering College. The reports are based on the Major project work done entirely by us and not copied from any other source. We submit our Major project for further development by any interested students who share similar interests to improve the Major project in the future.

The results embodied in this Major project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

|                           |                     |
|---------------------------|---------------------|
| <b>SOMA JHANSI</b>        | <b>(228R1A66C2)</b> |
| <b>GUNDA MAHITHA</b>      | <b>(228R1A6687)</b> |
| <b>AJMEERA SHRUTI</b>     | <b>(228R1A6668)</b> |
| <b>KANKANALA VASUDEVA</b> | <b>(228R1A6698)</b> |

## **ACKNOWLEDGEMENT**

We are extremely grateful to **Dr. A. Srinivasula Reddy**, Principal and **Dr. Madhavi Pingili**, Professor & HOD, Department of CSE(AI&ML), CMR Engineering College for their constant support.

We are extremely thankful to **Mrs.M.Srikala**, Assistant Professor, Internal Guide, Department of CSE(AI&ML), for her constant guidance, encouragement and moral support throughout the Major project.

We will be failing in duty if we do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Major project.

We thank **Mr. G. Venkateswarlu**, Major Project Coordinator for his constant support in carrying out the Major project activities and reviews.

We express our thanks to all staff members and friends for all the help and co-ordination extended in bringing out this Major project successfully in time.

Finally, We are very much thankful to our parents who guided us for every step.

|                           |                     |
|---------------------------|---------------------|
| <b>SOMA JHANSI</b>        | <b>(228R1A66C2)</b> |
| <b>GUNDA MAHITHA</b>      | <b>(228R1A6687)</b> |
| <b>AJMEERA SHRUTI</b>     | <b>(228R1A6668)</b> |
| <b>KANKANALA VASUDEVA</b> | <b>(228R1A6698)</b> |

# CONTENTS

| TOPIC                                   | PAGE NO   |
|-----------------------------------------|-----------|
| ABSTRACT                                | I         |
| LIST OF FIGURES                         | II        |
| LIST OF TABLES                          | III       |
| <b>1. INTRODUCTION</b>                  | <b>1</b>  |
| 1.1. Introduction                       | 1         |
| 1.2. Project Objectives                 | 1         |
| 1.3. Purpose of the project             | 2         |
| 1.4 Problem Statement                   | 2         |
| 1.5. Existing System with Disadvantages | 2         |
| 1.6. Proposed System with Advantages    | 3         |
| 1.7. Input and Output Design            | 4         |
| <b>2. LITERATURE SURVEY</b>             | <b>6</b>  |
| <b>3. SOFTWARE REQUIREMENT ANALYSIS</b> | <b>10</b> |
| 3.1. Modules and their Functionalities  | 10        |
| 3.2. Functional Requirements            | 11        |
| 3.3. Non-Functional Requirements        | 11        |
| 3.4. Feasibility Study                  | 12        |
| <b>4. SYSTEM SPECIFICATIONS</b>         | <b>14</b> |
| 4.1. Software requirements              | 14        |
| 4.2. Hardware requirements              | 14        |
| <b>5. SOFTWARE DESIGN</b>               | <b>15</b> |
| 5.1. System Architecture                | 15        |
| 5.2. Dataflow Diagrams                  | 17        |
| 5.3. UML Diagrams                       | 18        |

|                                     |           |
|-------------------------------------|-----------|
| <b>6. CODING AND IMPLEMENTATION</b> | <b>24</b> |
| 6.1. Source Code                    | 24        |
| 6.2. Implementation                 | 62        |
| <b>7. SYSTEM TESTING</b>            | <b>64</b> |
| 7.1. Types of System Testing        | 64        |
| 7.2. Test Strategies                | 66        |
| 7.3. Sample Test Cases              | 69        |
| <b>8. RESULTS</b>                   | <b>73</b> |
| <b>9. CONCLUSION</b>                | <b>76</b> |
| <b>10. FUTURE ENHANCEMENTS</b>      | <b>78</b> |
| <b>REFERENCES</b>                   | <b>80</b> |

# ABSTRACT

It is challenging to make precise assessments of real estate prices due to its elevated individual prices, complicated influencing factors, and ambiguous attribute selection. As a result of the high demand for owner-occupied and investment properties, real estate is also a substantial concern for society. A hot topic for research by major institutions has been how to accurately estimate its price. Realworld applications of real estate valuation impose stringent requirements on the acquisition of datasets and the generalizability of models. On the basis of SRGCNN, a spatial regression model with excellent generalizability, this paper introduces an external attention mechanism to construct the A-SRGCNN model and compares it to the benchmark model utilizing data from Shanghai, Melbourne, and San Diego. For spatial regression, A-SRGCNN employs graph convolutional neural networks, and the external attention mechanism implicitly considers the relationship between property data. Experiments indicate that the A-SRGCNN model outperforms the benchmark model and has improved real estate price estimation accuracy. In the meantime, this paper employs the A-SRGCNN model to conduct zonal experiments and time-division experiments on the secondary real estate market in Shanghai to analyze the real estate price linkages between different zones and the real estate price linkages at different times. It is revealed that Shanghai real estate prices exhibit spatial aggregation and price aggregation, with comparable prices within the same zones, and that the A-SRGCNN model is effective at predicting house prices.

**Keywords:** Real Estate Valuation; Graph Convolutional Network (GCN); Spatial Dependencies; Property Price Prediction; Machine Learning; Adjacency Matrix; Data Preprocessing; Artificial Intelligence.

## LIST OF FIGURES

| S.NO | FIGURE NO | DESCRIPTION                                    | PAGE NO |
|------|-----------|------------------------------------------------|---------|
| 1    | 1.5.1     | Block diagram of proposed system               | 4       |
| 2    | 5.1       | System Architecture                            | 15      |
| 3    | 5.2       | Data Flow diagram                              | 17      |
| 4    | 5.3.1     | Sequence diagram                               | 19      |
| 5    | 5.3.2     | Use case diagram                               | 21      |
| 6    | 5.3.3     | Activity diagram                               | 22      |
| 7    | 5.3.4     | Class diagram                                  | 23      |
| 8    | 7.3.1     | User Login                                     | 70      |
| 9    | 7.3.2     | User Registration                              | 70      |
| 10   | 7.3.3     | User Account Activation                        | 71      |
| 11   | 7.3.4     | Text-Based Prediction                          | 71      |
| 12   | 7.3.5     | Voice-Based Prediction                         | 72      |
| 13   | 7.3.6     | Admin Panel                                    | 72      |
| 14   | 8.1       | Social Media Forensics Landing Page            | 73      |
| 15   | 8.2       | User Authentication & System Access            | 73      |
| 16   | 8.3       | Ensemble-Based Text Classification             | 74      |
| 17   | 8.4       | Detection of Subtle Harassment via Voice Input | 74      |
| 18   | 8.5       | Voice Transcription & Forensic Analysis        | 75      |
| 19   | 8.6       | Statistical Data - Class Distribution          | 75      |

## **LIST OF TABLES**

| <b>S.NO</b> | <b>TABLE NO</b> | <b>DESCRIPTION</b>        | <b>PAGE NO</b> |
|-------------|-----------------|---------------------------|----------------|
| 1           | 2               | Literature Review Summary | 8-9            |
| 2           | 7.3             | Test Cases                | 69             |

# 1. INTRODUCTION

## 1.1 Introduction

Real estate valuation is a crucial component of the housing market, influencing decisions related to lending, investment, taxation, and urban planning, and therefore requires accurate and reliable models. Traditional methods such as hedonic pricing, regression techniques, and expert assessments have been widely used but often fail to capture the important spatial dependencies between properties that significantly impact value. Machine learning approaches, including linear regression, decision trees, random forests, and gradient boosting, have improved predictive performance; however, they still struggle to effectively model the complex spatial relationships inherent in real estate data. To address this limitation, Graph Convolutional Networks (GCNs) provide a powerful graph-based framework that represents properties as interconnected nodes, enabling the learning of spatial patterns and relationships.

The proposed GCN-based model preprocesses data through encoding and scaling, constructs an adjacency matrix to represent property connectivity, and applies graph convolutional layers to extract spatial features for improved prediction accuracy. This approach offers key advantages such as better incorporation of spatial dependencies, scalability, adaptability to multiple data sources, and the ability to perform well even with limited labeled data. Experimental results demonstrate that the GCN model outperforms traditional machine learning methods in accuracy and stability across datasets, highlighting its effectiveness and practical applicability in real-world real estate valuation.

## 1.2 Project Objectives

By the completion of this project, the system demonstrates the following capabilities:

1. Construct a spatial graph where nodes represent properties or neighborhoods in the megacity and edges encode spatial proximity or socio-economic relationships.
2. Integrate heterogeneous data sources such as property attributes, demographic information, transportation networks, and land-use data.
3. Evaluate how graph-based learning improves performance compared to traditional models such as linear regression, random forests, or gradient boosting.

### **1.3 Purpose of the Project**

The project aims to develop an accurate real estate valuation system using Graph Convolutional Networks (GCNs) that effectively capture spatial dependencies and relationships between properties. It improves prediction accuracy compared to traditional methods by representing real estate data as a graph with nodes and edges. The model incorporates geographic and socio-economic factors to better understand property value variations and is designed to handle complex urban data in megacities. Additionally, it provides scalable and data-driven predictions while reducing errors caused by treating properties independently, ultimately supporting better decision-making through an AI-based valuation framework.

### **1.4 Problem Statement**

Megacities experience rapid urbanization, leading to highly complex and dynamic real estate markets. Traditional valuation methods fail to capture spatial dependencies, non-linear property relationships, and neighborhood -level influences that significantly impact property prices. Existing models rely on linear assumptions or isolated property features, resulting in inaccurate predictions. The problem is to design a system that can integrate multi-dimensional property data, spatial proximity, socio-economic indicators, and urban infrastructure details to develop a highly accurate valuation model. The proposed solution utilizes Graph Convolutional Networks (GCNs) to learn both property-level features and spatial relationships, enabling better prediction accuracy for real estate valuation in megacity environments.

### **1.5 Existing System**

The existing real estate valuation systems mainly rely on traditional and machine learning approaches such as Hedonic Pricing Models (HPMs), Random Forest, Gradient Boosting, XGBoost, and spatial regression techniques. Hedonic models use linear regression based on property features like size, location, and amenities but fail to handle complex and non-linear relationships. Machine learning models improve prediction accuracy but treat properties as independent data points, ignoring spatial relationships.

Spatial regression methods attempt to include geographic dependencies but depend on predefined spatial weight matrices and often assume linearity. These approaches also struggle with incomplete or insufficient datasets, limiting their effectiveness. Additionally, they are not well-suited for capturing dynamic market conditions and real-time price variations. The models often lack flexibility and adaptability across different regions or cities. Their dependency on structured data further restricts their ability to incorporate diverse real-world factors.

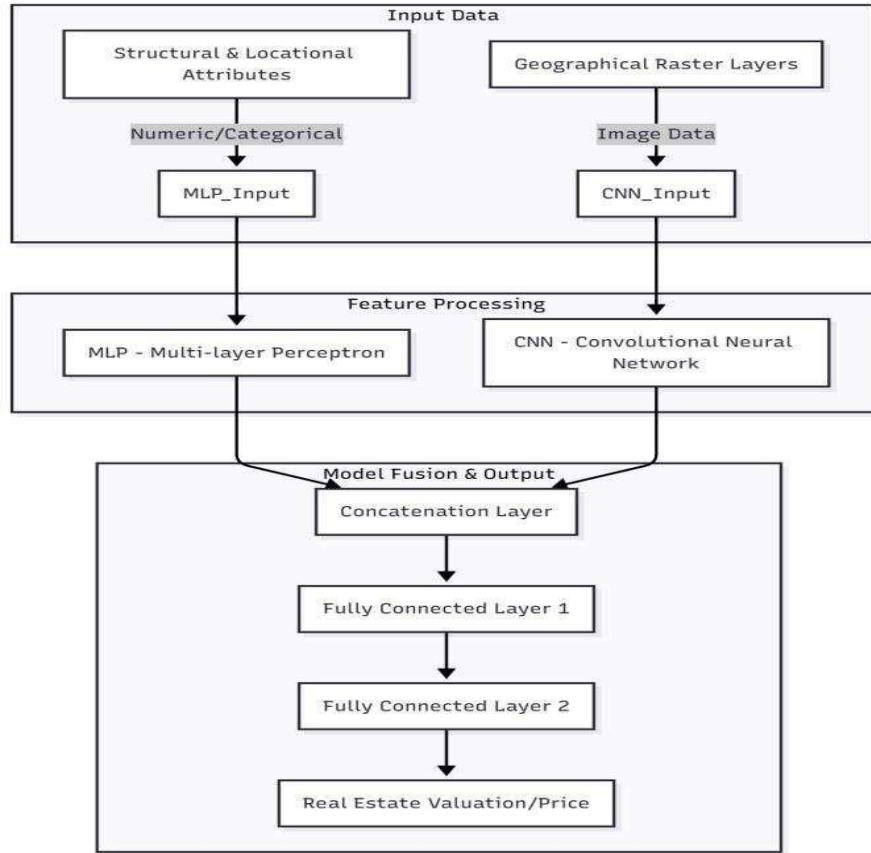
### **Disadvantages**

- Ignores spatial dependencies between properties.
- Limited generalization across different cities or regions.
- Inability to capture time-based price fluctuations.
- Requires large and complete labeled datasets.
- Assumes linear relationships and struggles with complex patterns.

## **1.6 Proposed System**

The proposed system introduces a Graph Convolutional Network (GCN)-based approach integrated with a Django-powered web application to improve real estate valuation. It models properties as interconnected nodes, effectively capturing spatial dependencies between neighboring properties. The system incorporates an external attention mechanism to identify hidden relationships and enhance model performance and generalization. It also uses semi-supervised learning, enabling the model to learn from both labeled and unlabeled data, making it suitable for limited datasets. A user-friendly web interface allows users to register, log in, predict property prices, and view past results. Admin functionalities include managing users, monitoring predictions, and controlling account access. The system is designed with a scalable, cloud-ready architecture for deployment on platforms like AWS and Google Cloud. It supports real-time predictions and dynamic data handling. Experimental results demonstrate superior accuracy compared to traditional models like Random Forest and XGBoost.

Overall, the system provides a robust, scalable, and intelligent solution for modern real estate valuation.



**Fig 1.5.1:** Block diagram of proposed system.

### Advantages

- Effectively captures spatial relationships using graph-based learning.
- Works well even with limited data through semi-supervised learning.
- Provides a user-friendly and scalable web-based interface.
- Achieves higher accuracy compared to traditional machine learning models.

## 1.7 Input and Output Design

### 1.7.1 Input Design

The input design for the Graph Convolutional Network-Based Model for Megacity Real Estate Valuation focuses on collecting, organizing, and preparing multi-dimensional data required for accurate property price prediction. The system accepts heterogeneous inputs such as property attributes, geographic coordinates, neighborhood characteristics, transportation accessibility, and socio-economic indicators. These inputs are transformed into a structured node feature matrix, where each property is represented by numerical and categorical features including size, age, location, amenities, and environmental factors.

To model spatial relationships between properties, the input design also constructs a **graph structure** in the form of an adjacency matrix, where edges reflect geographic proximity, road-network connections, or locality similarity. Before feeding data into the GCN, all inputs undergo preprocessing steps such as normalization, missing-value handling, encoding of categorical variables, and integration of GIS data. This input design ensures that both individual property characteristics and inter-property spatial dependencies are captured effectively, enabling the GCN model to learn complex urban patterns and deliver accurate real estate valuation in a megacity environment.

### Objectives

- A consistent and structured format has been established for capturing raw textual data, ensuring that all inputs are correctly received and prepared for preprocessing.
- The input pipeline effectively filters noise, ambiguity, and irrelevant elements, supporting a smooth transition into tokenization, feature extraction, and subsequent analytical stages.
- The system supports both multi-class and multi-label text inputs, enabling classification across diverse categories of cyberbullying and hate speech content.

### 1.7.2 Output Design

The output design of the Graph Convolutional Network-Based Model for Megacity Real Estate Valuation focuses on presenting the predicted property values in a clear, accurate, and user-interpretable format. After processing the input features and spatial relationships through the GCN layers, the system generates outputs such as estimated property prices, valuation confidence scores, and neighborhood influence metrics. T structured into an output layer that provides numerical predictions, which can be displayed through dashboards, tables, or real estate maps.

Additionally, the system may produce visual outputs like price distribution heatmaps, spatial trend graphs, and model-explainability insights that highlight the factors influencing each prediction. The output design ensures that users—including buyers, analysts, planners, and developers—receive meaningful, actionable, and easy-to-understand valuation information derived from complex graph-based computations, thereby supporting informed decision-making in megacity real estate markets.

## 2. LITERATURE SURVEY

1. **Wang et al., “Multi-Source Geographic Big Data and Deep Neural Network for Smart Property Pricing,” in ICMPC Journal, 2025.** This study proposes a spatial attention-based deep neural network that integrates geographic, environmental, and economic features for accurate housing price prediction. It handles complex urban variations, incorporates infrastructure and socio-economic factors, and improves decision-making for smart cities while addressing overfitting using data augmentation techniques.
2. **Li et al., “Graph-Based Deep Learning for Assessing Urban Real Estate Reform Impacts,” in ACM Digital Library, 2025.** This work presents a graph neural network framework to analyze the impact of urban reforms and green-space developments on property prices. It highlights spatial relationships between properties and provides visualization of price influence zones, demonstrating robustness across multiple metropolitan regions.
3. **Chen et al., “Scalable Property Valuation Using Graph Convolution and Transformer-Based Networks,” in IEEE/ArXiv, 2024.** This paper introduces a hybrid GCN and Transformer model capable of handling large-scale property datasets with high spatial resolution. It supports multimodal learning, improves computational efficiency through sparse encoding, and reduces bias through fairness-aware evaluation strategies.
4. **Zhang et al., “Temporal Graph Networks for Cross-City Real Estate Appraisal,” in ArXiv, 2024.** This research focuses on temporal graph networks combined with transfer learning to enable real estate model adaptation across cities. It captures time-based price trends and seasonal variations while reducing retraining costs and improving generalization in diverse urban environments.
5. **Nguyen et al., “Do Graph Neural Networks Really Improve House Price Prediction?” in 2024.** This study benchmarks GNNs against models like XGBoost and LightGBM, showing that GNNs perform better when spatial and neighborhood information is properly modeled. It emphasizes the importance of graph construction techniques and suggests hybrid GNN-based models for improved stability.
6. **Yang et al., “Graph Convolutional Network-Based Model for Megacity Real Estate Valuation,” in IEEE Access, 2022.** This work introduces the A-SRGCNN model, which combines graph convolution with an external attention mechanism to improve prediction accuracy. It captures spatial clustering effects and achieves superior performance compared to traditional machine learning approaches.

7. **Zhu et al., “Spatial Regression Graph Convolutional Neural Networks (SRGCNN),” in GeoInformatica Journal, 2021.** This paper replaces traditional spatial regression with graph convolution techniques to model complex geospatial dependencies. It supports semi-supervised learning for handling missing data and improves learning stability, forming the foundation for advanced spatial deep learning models.
8. **Law et al., “Image-Driven Housing Price Estimation Using Street View and Satellite Imagery,” in ACM TIST, 2019.** This study uses convolutional neural networks to analyze visual features such as street view and satellite images for property valuation. It demonstrates that environmental aesthetics influence pricing and promotes multimodal approaches combining visual and structured data.
9. **Bin et al., “RNN-Boosting Fusion for Real Estate Appraisal,” in IEEE Conference, 2017.** This research combines recurrent neural networks with boosting techniques to capture temporal trends and seasonal price variations. The hybrid approach improves prediction accuracy during market fluctuations and enhances adaptability across different city datasets.
10. **Huang et al., “Geographically Weighted Regression for Spatio-Temporal House Price Modeling,” in International Journal of GIS, 2010.** This study introduces geographically weighted regression to capture spatial heterogeneity in property prices. It highlights the importance of location-based variations and serves as a foundational model for later spatial and deep learning-based valuation methods.

| <b>Focused Area / Title</b>                               | <b>Key Findings</b>                                                                                                                                                                                                 | <b>Reference</b>                                                                                                         |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Multi-Source Geographic Big Data for Property Pricing [1] | Proposes a spatial attention-based deep neural network integrating geographic, environmental, and economic features. Improves accuracy in urban property valuation and reduces overfitting using data augmentation. | Wang et al., “Multi-Source Geographic Big Data and Deep Neural Network for Smart Property Pricing,” ICMPC Journal, 2025. |
| Graph-Based Real Estate Reform Impact Analysis [2]        | Li et al., “Graph-Based Deep Learning for Assessing Urban Real Estate Reform Impacts,” ACM Digital Library, 2025.                                                                                                   | Li et al., “Graph-Based Deep Learning for Assessing Urban Real Estate Reform Impacts,” ACM Digital Library, 2025.        |
| Scalable GCN + Transformer Property Valuation [3]         | Introduces a hybrid GCN and Transformer model for large-scale datasets. Enhances computational efficiency, supports multimodal learning, and reduces bias in predictions..                                          | Chen et al., “Scalable Property Valuation Using Graph Convolution and Transformer-Based Networks,” IEEE/ArXiv, 2024.     |
| Temporal Graph Networks for Cross-City Valuation [4]      | Applies temporal graph networks and transfer learning for cross-city real estate prediction. Captures seasonal trends and improves generalization across regions..                                                  | Zhang et al., “Temporal Graph Networks for Cross-City Real Estate Appraisal,” ArXiv, 2024.                               |
| GNN vs Traditional Models for Price Prediction [5]        | Benchmarks GNNs against XGBoost and LightGBM. Shows GNNs perform better when spatial relationships are modeled properly and highlights importance of graph construction.                                            | Nguyen et al., “Do Graph Neural Networks Really Improve House Price Prediction?”, 2024.                                  |

**Table no. 2** Literature Review Summary

| <b>Focused Area / Title</b>                             | <b>Key Findings</b>                                                                                                                                                       | <b>Reference</b>                                                                                                                 |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| GCN-Based Megacity Real Estate Valuation (A-SRGCNN) [6] | Combines graph convolution with external attention to improve prediction accuracy. Captures spatial clustering and achieves superior performance over traditional models. | Yang et al., “Graph Convolutional Network-Based Model for Megacity Real Estate Valuation,” IEEE Access, 2022.                    |
| Spatial Regression GCN (SRGCNN)[7]                      | Replaces traditional spatial regression with graph convolution for modeling geospatial dependencies. Supports semi-supervised learning and improves scalability.          | Zhu et al., “Spatial Regression Graph Convolutional Neural Networks (SRGCNN),” GeoInformatica Journal, 2021..                    |
| Image-Based Property Valuation Using CNN[8]             | Uses street view and satellite imagery with CNNs to estimate property prices. Demonstrates that visual surroundings significantly influence valuation                     | Law et al., “Image-Driven Housing Price Estimation Using Street View and Satellite Imagery,” ACM TIST, 2019                      |
| RNN-Boosting Hybrid Model for Real Estate [9]           | Combines RNN with boosting to capture temporal price trends and seasonal variations. Improves prediction during market fluctuations.                                      | Bin et al., “RNN-Boosting Fusion for Real Estate Appraisal,” IEEE Conference, 2017                                               |
| Geographically Weighted Regression (GWR) [10]           | Introduces spatial regression capturing location-based price variations. Serves as a baseline model and highlights need for advanced spatial learning techniques.         | Huang et al., “Geographically Weighted Regression for Spatio-Temporal House Price Modeling,” International Journal of GIS, 2010. |

**Table no. 2** Literature Review Summary

## **3. SOFTWARE REQUIREMENTS ANALYSIS**

### **3.1 Modules and Their Functionalities**

#### **3.1.1 Data Analysis**

The data analysis phase involves examining real estate datasets collected from multiple cities to understand patterns influencing property prices. The dataset includes features such as location, property size, amenities, and socio-economic indicators. Data preprocessing is carried out to handle missing values, encode categorical variables, and normalize numerical features for consistency. Exploratory analysis shows that location and neighborhood characteristics have a significant impact on property valuation.

Correlation analysis is used to identify the most influential features affecting price prediction. Spatial relationships between properties are analyzed using adjacency matrices to capture neighborhood dependencies. The data distribution is also studied to detect imbalances and improve model training. Visualization techniques like heatmaps and scatter plots help in understanding feature relationships clearly. The dataset is divided into training and testing sets for proper evaluation of the model.

#### **3.1.2 Data Preprocessing**

The data preprocessing stage prepares the real estate dataset for effective model training by ensuring data quality and consistency. It involves handling missing values and removing irrelevant or noisy data entries. Categorical variables such as location and property type are converted into numerical form using label encoding. Numerical features are scaled using standard normalization techniques to maintain uniformity across inputs. An adjacency matrix is constructed to represent spatial relationships between properties for graph-based learning. Data is also cleaned and structured to support both tabular and spatial inputs.

#### **3.1.3 Deep Learning Algorithm for Prediction**

The project uses Graph Convolutional Networks (GCNs) for real estate price prediction, where properties are represented as nodes and their spatial relationships are modeled as edges.

An adjacency matrix is used to define connections based on geographic proximity, allowing the model to capture neighborhood influences. The GCN learns spatial dependencies through message passing between connected nodes. Additionally, an external attention mechanism and semi-supervised learning approach enhance the model's performance and accuracy. The model is trained using historical data and optimized with RMSE to provide accurate and scalable predictions.

### **3.2 Functional Requirements**

The system should allow users to register and log in securely to access the application. It must enable users to input property details and generate real-time price predictions using the trained model. The system should store and display previous prediction results for user reference. Admin functionalities should include managing user accounts, monitoring system activity, and controlling access. Additionally, the system should process input data, perform predictions using the GCN model, and return accurate results efficiently.

- User registration and secure login functionality.
- Input interface for property details and prediction request.
- Real-time property price prediction using the model.
- Storage and retrieval of past prediction results.

### **3.3 Non-Functional Requirements**

The system should ensure high performance by providing fast and accurate real-time predictions. It must be scalable to handle multiple users simultaneously, especially when deployed on cloud platforms. Security is essential, requiring secure user authentication and protection of sensitive data. The system should maintain high reliability and availability to ensure uninterrupted service. Additionally, it should offer a user-friendly interface and maintainability for easy updates and future enhancements.

- High performance with quick response time for predictions.
- Scalability to support a large number of concurrent users.
- Strong security for data protection and user authentication.

- High reliability and system availability.

### **3.4 Feasibility Study**

The feasibility study evaluates the practicality and effectiveness of the proposed real estate valuation system. It analyzes whether the system can be successfully implemented using available resources and technologies. The study considers economic, technical, and operational aspects to ensure overall viability. It also identifies potential risks and limitations in system development and deployment. Overall, the feasibility study confirms that the proposed GCN-based system is practical and beneficial for real-world applications.

1. Economic Feasibility
2. Technical Feasibility
3. Operational Feasibility

#### **3.4.1 Economic Feasibility**

The proposed system is economically feasible as it utilizes open-source tools and technologies, reducing overall development costs. Cloud-based deployment further minimizes the need for expensive hardware infrastructure. The automation of real estate valuation helps reduce manual effort and associated operational expenses. Maintenance costs are also low due to the system's scalable and efficient design. The system can be easily upgraded without significant additional investment. Overall, it provides a cost-effective solution with long-term financial benefits.

#### **3.4.2 Technical Feasibility**

The system is technically feasible with the use of advanced technologies like Graph Convolutional Networks and Django framework. Required tools such as Python, machine learning libraries, and cloud platforms are readily available. The model is capable of handling large datasets and complex spatial relationships efficiently. Integration between frontend, backend, and machine learning components is achievable. The system supports scalability and can be optimized for real-time performance. Overall, the technical requirements can be successfully implemented with current technologies.

### **3.4.3 Operational Feasibility**

The system is operationally feasible as it offers a simple and user-friendly interface for both users and administrators. Users can easily input property details and obtain predictions without technical difficulty. Admin functionalities allow effective monitoring and management of system operations. The system requires minimal training for users to operate efficiently. It supports real-time usage, making it suitable for practical applications. Overall, it enhances usability and improves decision-making processes in real estate valuation..

## 4. SYSTEM SPECIFICATIONS

### 4.1 Software Requirements

The software stack for this project includes programming environments, deep learning libraries, graph processing frameworks, GIS tools, and visualization platforms. Python (version 3.8 or above) serves as the core programming environment because of its flexibility, extensive library support, and ease of integration with machine learning workflows. Deep learning frameworks such as PyTorch or TensorFlow provide the computational foundation needed for training GCN models with GPU acceleration. For handling graph-based operations, libraries like PyTorch Geometric (PyG) or the Deep Graph Library (DGL) are essential as they support graph construction, message passing, and adjacency matrix generation.

- Operating System: Windows / Linux / macOS
- Programming Language: Python 3.8 or above
- Core Libraries: Scikit-learn, NumPy, Pandas
- Development Environment: VS Code / PyCharm / Jupyter Notebook / GitHub
- Documentation Tools: My SQL/ Postgre SQL/SQLite

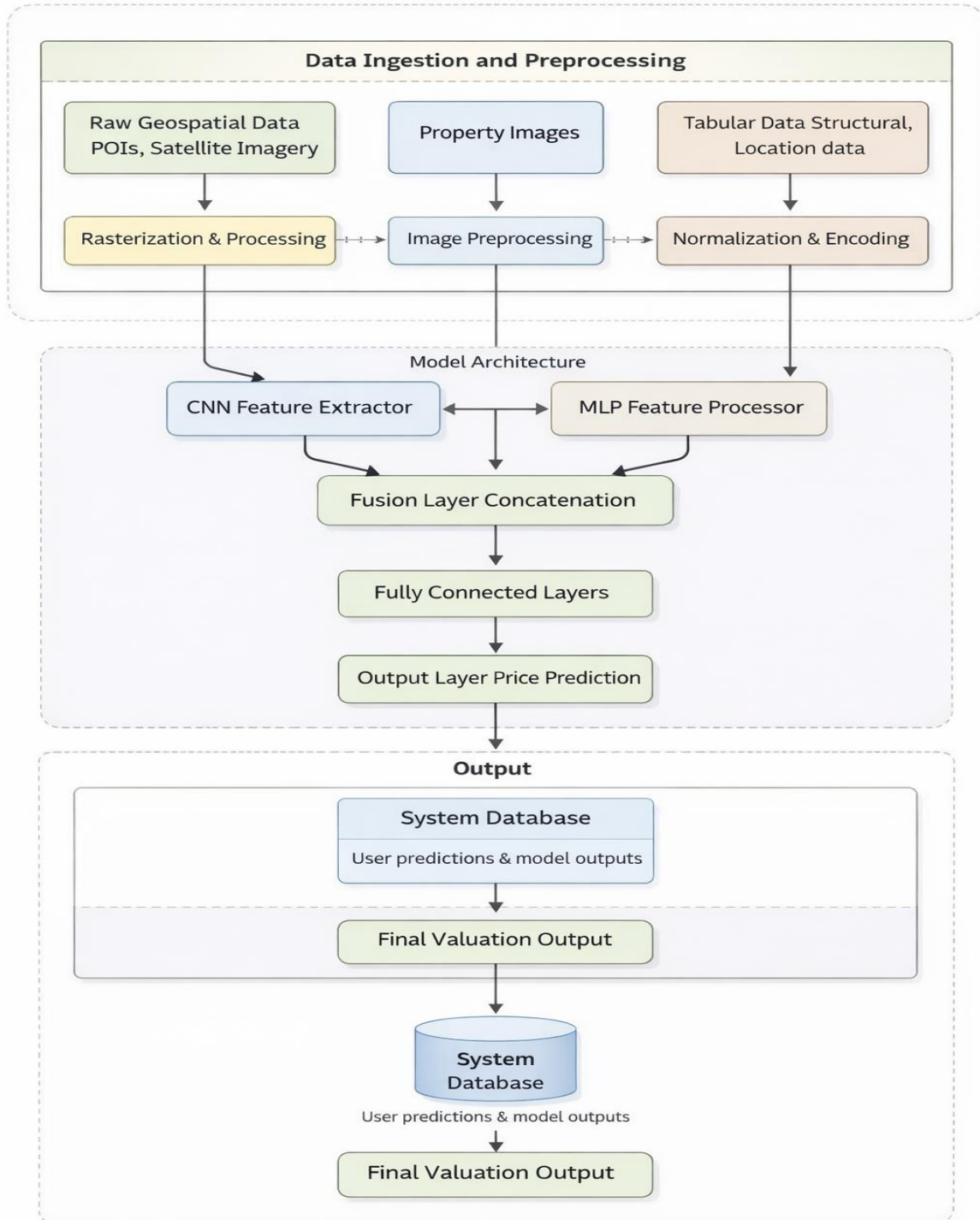
### 4.2 Hardware Requirements

The hardware requirements for building the GCN-based real estate valuation system depend on the scale and complexity of the dataset. For small datasets and preliminary experiments, a basic system with an Intel Core i5 processor, 8 GB RAM, and a standard GPU or integrated graphics may suffice. However, training GCN models on larger megacity datasets requires more robust hardware. A recommended system includes an Intel Core i7/i9 or AMD Ryzen 7/9 processor, 16–32 GB RAM, and an NVIDIA RTX series GPU (such as RTX 3060 or 3070) with at least 6–8 GB of VRAM, enabling efficient graph processing and faster training times. SSD storage ranging from 512 GB to 1 TB is preferred for quick data access and model loading.

- Processor: Intel Core i5(7th Gen)/AMD equivalent
- Memory (RAM): Minimum 8 GB
- Storage: 250 GB HDD/SSD
- Display: Standard 14" or higher
- Optional: Windows 10 / Linux Ubuntu

## 5. SOFTWARE DESIGN

### 5.1 System Architecture



**Fig: 5.1** System Architecture

The system architecture is designed to provide an efficient and scalable framework for real estate valuation by integrating multiple data sources and advanced learning techniques. It begins with the data ingestion layer, where raw geospatial data such as points of interest and satellite imagery, property images, and structured tabular data are collected. These diverse data types ensure that both spatial and attribute-based information are captured effectively. In the preprocessing stage, geospatial data undergoes rasterization and processing to convert it into usable formats. Property images are preprocessed through resizing, normalization, and enhancement techniques. Similarly, tabular data is cleaned, normalized, and encoded to prepare it for model input.

The processed data is then passed into the model architecture, which consists of multiple feature extraction components. A Convolutional Neural Network (CNN) is used to extract deep features from image and spatial data. At the same time, a Multi-Layer Perceptron (MLP) processes structured tabular features such as location and property attributes. These extracted features from different modalities are combined in a fusion layer using concatenation. This fusion step enables the model to integrate both visual and structured information for better prediction performance.

The combined features are then passed through fully connected layers, which learn complex patterns and relationships in the data. The output layer generates the predicted property price based on the learned features. This prediction is then forwarded to the output layer of the system. The results are stored in the system database, which maintains user predictions and model outputs for future reference. Finally, the system provides the final valuation output to the user. Overall, this architecture ensures accurate, scalable, and real-time real estate price prediction by leveraging multimodal data and deep learning techniques.

## 5.2 Dataflow Diagram

The Level-1 Data Flow Diagram represents the overall functioning of the ensemble classification system by illustrating interactions between external entities, processes, and the database. The two primary entities are the User and Admin, both interacting through the User Authentication process. The user provides login credentials and receives authentication status, while the admin can validate requests and access the user list. The user provides login credentials and receives authentication status, while the admin can validate requests and access the user list.

Once authenticated, the user submits raw text input, which flows through the core processing stages. The Text Preprocessing module cleans and standardizes the input, followed by Feature Extraction that converts the text into numerical feature vectors. These vectors are then passed to the Ensemble Classification module, where Bagging Random Forest and Boosted Decision Tree models generate predictions.

The prediction results are stored in the Application Database along with user information. The Result Display and Management module retrieves stored predictions and presents them to the user. Additionally, the admin can view stored predictions directly from the database, enabling monitoring and system oversight.

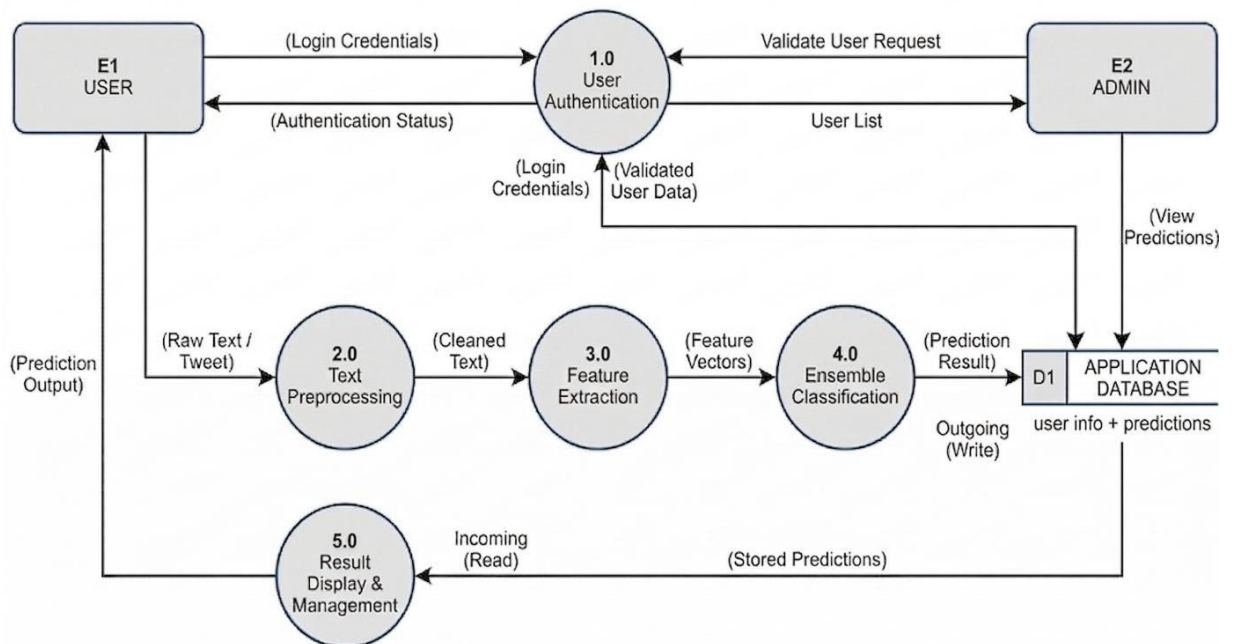


Fig 5.2 Dataflow Diagram

## 5.3 UML Diagrams

UML (Unified Modeling Language) is a standardized language used for specifying, visualizing, constructing, and documenting the artifacts of software systems. UML helps in representing the design of systems and understanding their components. Created by the Object Management Group (OMG), UML 1.0 was proposed in January 1997. UML is closely associated with object-oriented analysis and design. The two main categories of UML diagrams are Behavioral and Structural diagrams, each serving distinct purposes in the modeling process.

The Behavioral UML diagrams describe the behavior of the system, its actors, and the interaction between the components. On the other hand, Structural UML diagrams depict the static structure of the system, showing its components and relationships. UML has been integrated as a standard by OMG, and its primary goals are to provide a formal basis for understanding modeling languages, offer a ready-to-use expressive language for system developers, and encourage the growth of object-oriented tools.

### **Goals of UML:**

- To provide a standard visual representation of the system design.
- To simplify understanding of system architecture for developers and reviewers.
- To improve communication among team members during development.
- To model both structural and behavioral aspects of the system.
- To support object-oriented design and development practices.
- To document the system clearly for future reference and maintenance.

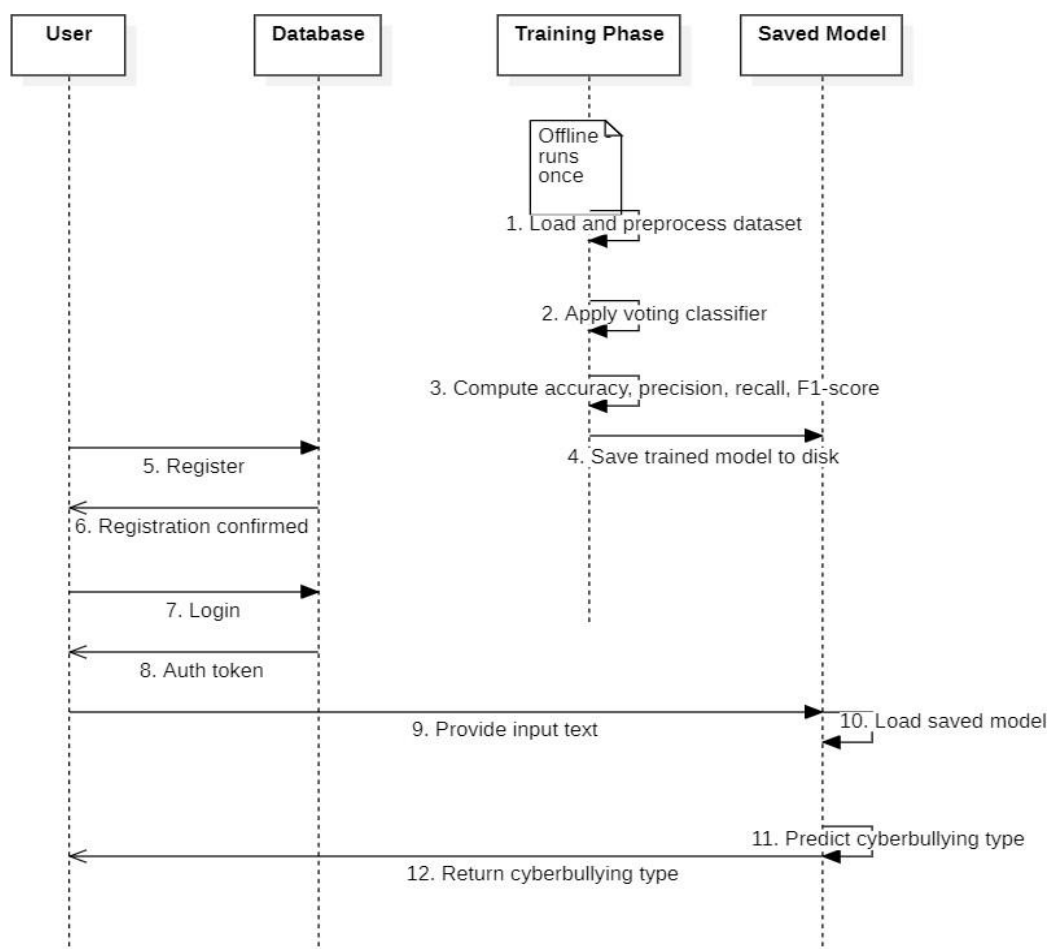
### **Types of UML Diagrams:**

1. Sequence Diagram:
2. Use Case Diagram:
3. Activity Diagram:
4. Class Diagram:

### 5.3.1. Sequence Diagram

The sequence diagram illustrates the end-to-end workflow of the cyberbullying detection system, covering both offline training and online inference processes. In the training phase, the system loads and preprocesses the dataset, applies the ensemble voting classifier combining Bagging Random Forest and Boosted Decision Tree, and evaluates performance using metrics such as accuracy, precision, recall, and F1-score. Once validated, the trained model is saved to disk as a persistent artifact. This phase is executed offline and ensures that the model is optimized and ready for deployment without affecting runtime performance.

In the online inference phase, the user interacts with the system by registering and logging in, with all credentials managed through the database. After authentication, the user provides input text, which triggers the system to load the saved model and perform prediction. The model analyzes the input and classifies it into the appropriate cyberbullying category. The prediction result is then returned to the user, completing the interaction cycle. This separation of training and inference enhances system efficiency, scalability, and real-time responsiveness.



**Fig 5.3.1:** Sequence Diagram

## **List of actions**

- **User:**

The user interacts with the system by registering and logging in through the interface. After successful authentication, the user provides input text for cyberbullying detection. The user finally receives the predicted classification result generated by the system.

- **Database:**

The user interacts with the system by registering and logging in through the interface. After successful authentication, the user provides input text for cyberbullying detection. The user finally receives the predicted classification result generated by the system.

- **Training Phase (Offline):**

The training phase runs offline, where the dataset is loaded and preprocessed. The ensemble voting classifier (Bagging Random Forest and Boosted Decision Tree) is applied, and performance metrics such as accuracy, precision, recall, and F1-score are computed. The trained model is then saved to disk for future use.

- **Saved Model:**

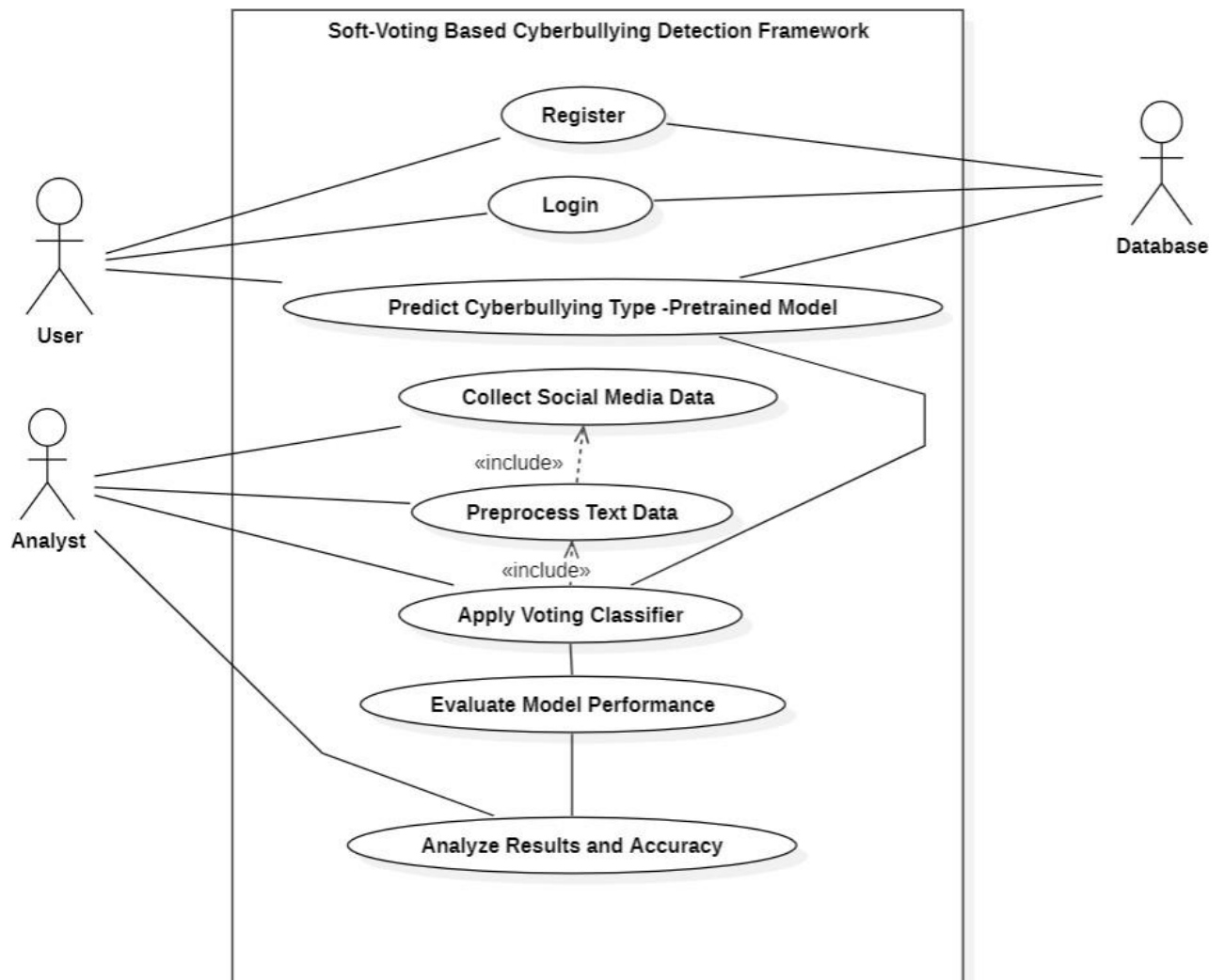
During runtime, the saved model is loaded when the user provides input text. The model processes the input and predicts the cyberbullying category. The prediction result is then returned to the user, completing the inference workflow.

### **5.3.2 Use Case Diagram**

The use case diagram illustrates the overall functionality of the soft-voting based cyberbullying detection framework by clearly distinguishing between the roles of the User and the Analyst. The user interacts with the system through registration and login, which are connected to the database for storing and validating credentials. After authentication, the user can directly access the prediction functionality, where input text is analyzed using a pretrained model to classify cyberbullying content. This represents the inference phase of the system, ensuring efficient and real-time prediction without involving the training process.

The Analyst is responsible for the training and evaluation pipeline of the system. This includes collecting social media data, preprocessing the text, and applying the voting classifier that combines Bagging Random Forest and Boosted Decision Tree models. The processed data is then used to evaluate model performance using standard metrics such as accuracy, precision, recall, and F1-score.

Finally, the analyst analyzes the results to assess model effectiveness and identify areas for improvement. This separation of training and inference ensures modularity, scalability, and clarity in system design.

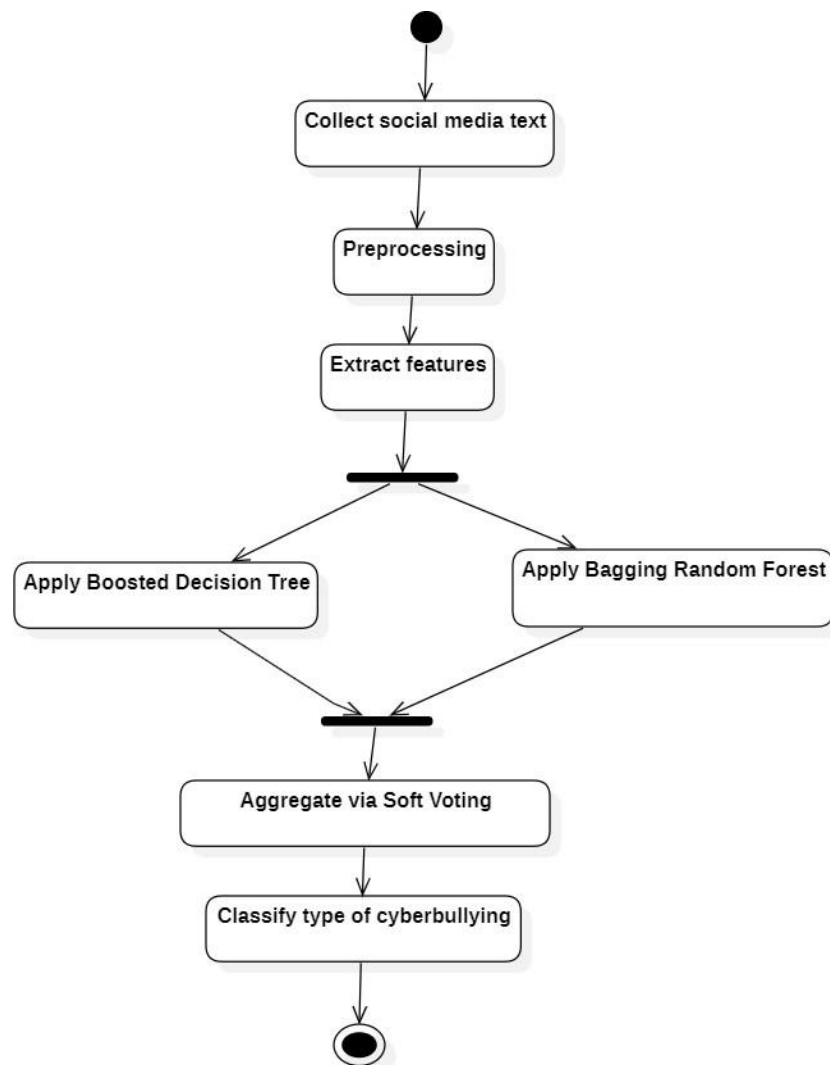


**Fig 5.3.2** Use Case Diagram

### 5.3.3 Activity Diagram

The activity diagram represents the step-by-step workflow of the soft-voting based cyberbullying detection system. It starts with collecting social media text, followed by preprocessing and feature extraction to transform raw input into structured features suitable for machine learning models. The flow then splits into two parallel activities where Boosted Decision Tree and Bagging Random Forest classifiers are applied simultaneously.

After parallel processing, the outputs from both models are merged and combined using a soft voting mechanism. This aggregation step enhances prediction reliability by considering the probabilities of both classifiers. Finally, the system classifies the type of cyberbullying and reaches the end state, completing the activity flow.



**Fig 5.3.3** Activity Diagram

### 5.3.4. Class Diagram

The class diagram represents the structural design of the cyberbullying detection system, showing the interaction between different components involved in training, prediction, and data management. The ModelTrainer trains individual models using the dataset, while the VotingClassifierBuilder aggregates these models and saves the final ensemble as a SavedModelFile. The Evaluation component assesses model performance using test data and generates reports, ensuring the model's effectiveness before deployment.

On the user side, the User interacts with the system by registering, logging in, and requesting classification. The CyberbullyingClassifier loads the pretrained model and processes user input to generate predictions, which are returned as a Result. The Database manages user credentials and stores prediction outputs, enabling authentication and persistence. Overall, the diagram clearly separates training, evaluation, and inference components, reflecting a modular and scalable system architecture.

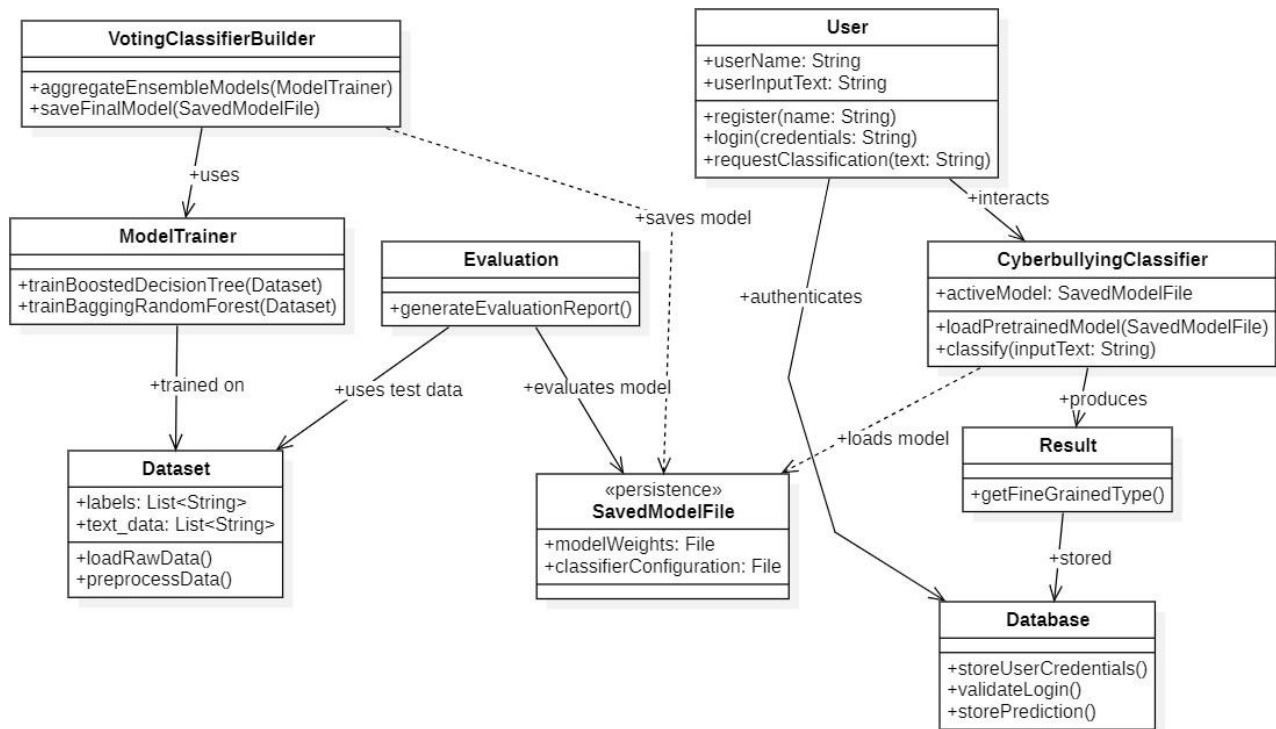


Fig 5.3.4 Class Diagram

## 6.CODING AND IMPLEMENTATION

### 6.1 Source Code

#### Dataset1.ipynb:

```
import warnings
warnings.filterwarnings('ignore')
```

```
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
data = pd.read_csv('dataset1/cyberbullying_tweets.csv')
```

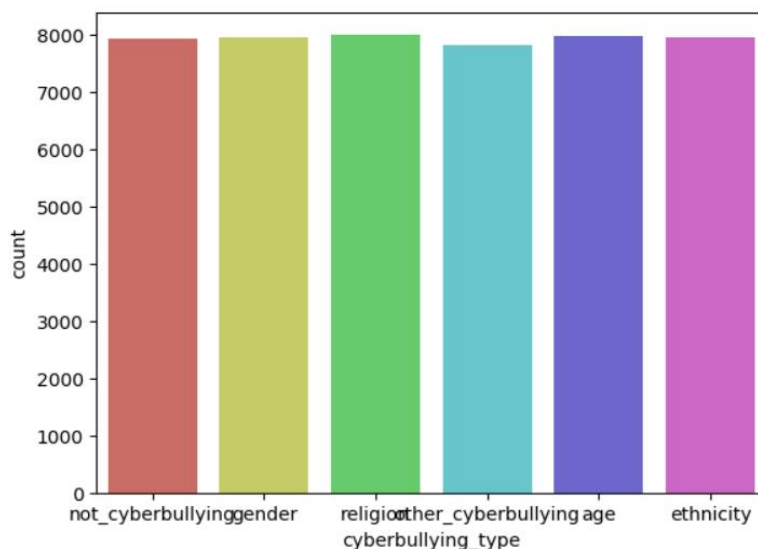
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 47692 entries, 0 to 47691
Data columns (total 2 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   tweet_text            47692 non-null  object
 1   cyberbullying_type    47692 non-null  object
dtypes: object(2)
memory usage: 745.3+ KB
```

```
data.head()
```

|   | tweet_text                                        | cyberbullying_type |
|---|---------------------------------------------------|--------------------|
| 0 | In other words #katandandre, your food was cra... | not_cyberbullying  |
| 1 | Why is #aussietv so white? #MKR #theblock #ImA... | not_cyberbullying  |
| 2 | @XochitiSuckkks a classy whore? Or more red ve... | not_cyberbullying  |
| 3 | @Jason_Gio meh. :P thanks for the heads up, b...  | not_cyberbullying  |
| 4 | @RudhoeEnglish This is an ISIS account pretend... | not_cyberbullying  |

```
sns.countplot(x='cyberbullying_type',data=data, palette='hls')
plt.show()
#plt.savefig('count_plot') mal: the nodule malignancy, 0: benign, 1: malignant
```





```

▶ from nltk.corpus import stopwords
  from sklearn.model_selection import train_test_split
  from nltk.tokenize import word_tokenize
  import string
  from nltk.stem import WordNetLemmatizer

```

```
data['cyberbullying_type'].value_counts()
```

```

religion      7998
age           7992
gender        7973
ethnicity     7961
not_cyberbullying 7945
other_cyberbullying 7823
Name: cyberbullying_type, dtype: int64

```

## Binary

```

def change_label(df):
    df['cyberbullying_type'].replace(['religion', 'age', 'gender', 'ethnicity', 'other_cyberbullying'], 'cyberbullying', inplace=True)

```

```
change_label(data)
```

```
data['cyberbullying_type'].value_counts()
```

```

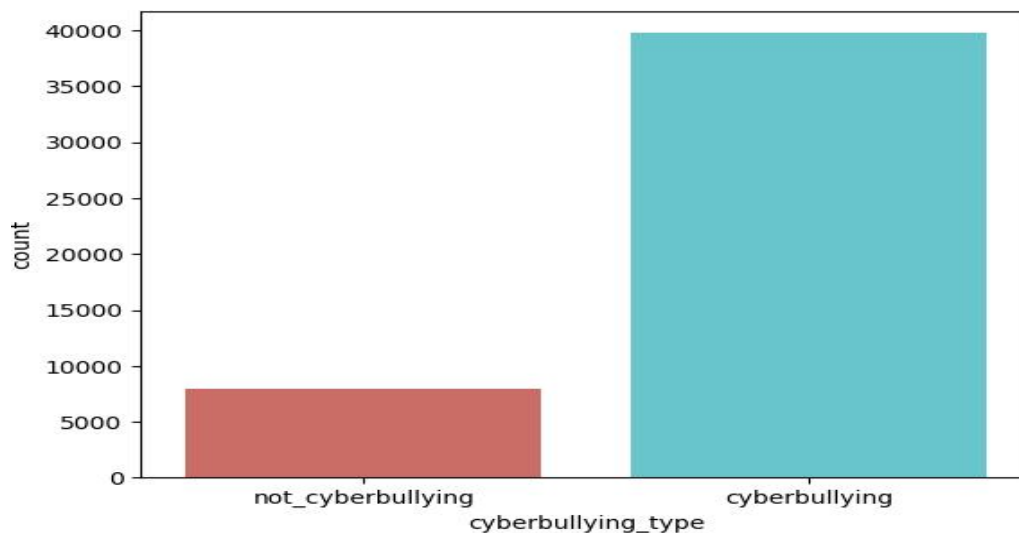
cyberbullying      39747
not_cyberbullying   7945
Name: cyberbullying_type, dtype: int64

```

```

sns.countplot(x='cyberbullying_type', data=data, palette='hls')
plt.show()

```



```

Tweet = []
Labels = []

for row in data["message"]:
    #tokenize words
    words = word_tokenize(row)
    #remove punctuations
    clean_words = [word.lower() for word in words if word not in set(string.punctuation)]
    #remove stop words
    english_stops = set(stopwords.words('english'))
    characters_to_remove = ["'", '"', "rt", "https", ":", "(", ")", "\u200b", "--", "n't", "'s", "...", "//t.c" ]
    clean_words = [word for word in clean_words if word not in english_stops]
    clean_words = [word for word in clean_words if word not in set(characters_to_remove)]
    #Lematise words
    wordnet_lemmatizer = WordNetLemmatizer()
    lemma_list = [wordnet_lemmatizer.lemmatize(word) for word in clean_words]
    Tweet.append(lemma_list)

```

```

# Import label encoder
from sklearn import preprocessing

# label_encoder object knows
# how to understand word labels.
label_encoder = preprocessing.LabelEncoder()

# Encode labels in column 'species'.
data['cyberbullying_type'] = label_encoder.fit_transform(data['cyberbullying_type'])

data['cyberbullying_type'].unique()

array([0, 1])

```

```

X = data['message']
y = data['cyberbullying_type']

```

```

# Extract Feature With CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
cv = TfidfVectorizer(stop_words='english')
X = cv.fit_transform(X) # Fit the Data

from imblearn.over_sampling import SMOTE

smt = SMOTE()
X, y = smt.fit_resample(X, y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

ML_Model = []
accuracy = []
precision = []
recall = []
f1score = []

#function to call for storing the results
def storeResults(model, a,b,c,d):
    ML_Model.append(model)
    accuracy.append(round(a, 3))
    precision.append(round(d, 3))
    recall.append(round(c, 3))
    f1score.append(round(b, 3))

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

```

## SVM

---

```
from sklearn import svm
svc = svm.LinearSVC()
svc.fit(X_train, y_train)

y_pred = svc.predict(X_test)

svc_acc = accuracy_score(y_pred, y_test)
svc_prec = precision_score(y_pred, y_test, average='weighted')
svc_rec = recall_score(y_pred, y_test, average='weighted')
svc_f1 = f1_score(y_pred, y_test, average='weighted')

storeResults('SVM', svc_acc, svc_prec, svc_rec, svc_f1)
```

---

## Random Forest

---

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier()

rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

rf_acc = accuracy_score(y_pred, y_test)
rf_prec = precision_score(y_pred, y_test, average='weighted')
rf_rec = recall_score(y_pred, y_test, average='weighted')
rf_f1 = f1_score(y_pred, y_test, average='weighted')

storeResults('RandomForest', rf_acc, rf_prec, rf_rec, rf_f1)
```

---

## Logistic Regression

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(multi_class = 'multinomial', solver = 'lbfgs')
lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)

lr_acc = accuracy_score(y_pred, y_test)
lr_prec = precision_score(y_pred, y_test, average='weighted')
lr_rec = recall_score(y_pred, y_test, average='weighted')
lr_f1 = f1_score(y_pred, y_test, average='weighted')

storeResults('LogisticRegression', lr_acc, lr_prec, lr_rec, lr_f1)
```

## MLP - OVO

```
from sklearn.multiclass import OneVsOneClassifier
from sklearn.neural_network import MLPClassifier

mlp = OneVsOneClassifier(MLPClassifier(random_state=1, max_iter=300))

mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)

mlp_acc = accuracy_score(y_pred, y_test)
mlp_prec = precision_score(y_pred, y_test, average='weighted')
mlp_rec = recall_score(y_pred, y_test, average='weighted')
mlp_f1 = f1_score(y_pred, y_test, average='weighted')

storeResults('MLP-OVO', mlp_acc, mlp_prec, mlp_rec, mlp_f1)
```

## MLP - OVR

```
from sklearn.multiclass import OneVsRestClassifier
mlp = OneVsRestClassifier(MLPClassifier(random_state=1))

mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)

mlp1_acc = accuracy_score(y_pred, y_test)
mlp1_prec = precision_score(y_pred, y_test, average='weighted')
mlp1_rec = recall_score(y_pred, y_test, average='weighted')
mlp1_f1 = f1_score(y_pred, y_test, average='weighted')

storeResults('MLP-OVR', mlp1_acc, mlp1_prec, mlp1_rec, mlp1_f1)
```

## Extension

```
from sklearn.ensemble import VotingClassifier, BaggingClassifier, ExtraTreesClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier

brf = BaggingClassifier(RandomForestClassifier())

bdt = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1))

model = VotingClassifier(estimators= [('BoostDT', bdt),('BagRF', brf)], voting='soft')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

ext_acc = accuracy_score(y_pred, y_test)
ext_prec = precision_score(y_pred, y_test,average='weighted')
ext_rec = recall_score(y_pred, y_test,average='weighted')
ext_f1 = f1_score(y_pred, y_test,average='weighted')
```

```
storeResults('Extension',ext_acc,ext_prec,ext_rec,ext_f1)
```

## Comparison

```
#creating dataframe
result = pd.DataFrame({ 'ML Model' : ML_Model,
                        'Accuracy' : accuracy,
                        'Precision': precision,
                        'Recall'   : recall,
                        'F1_score' : f1score
                        })
```

result

|   | ML Model           | Accuracy | Precision | Recall | F1_score |
|---|--------------------|----------|-----------|--------|----------|
| 0 | SVM                | 0.963    | 0.963     | 0.963  | 0.965    |
| 1 | RandomForest       | 0.954    | 0.954     | 0.954  | 0.955    |
| 2 | LogisticRegression | 0.943    | 0.943     | 0.943  | 0.946    |
| 3 | MLP-OVO            | 0.950    | 0.950     | 0.950  | 0.952    |
| 4 | MLP-OVR            | 0.950    | 0.950     | 0.950  | 0.952    |
| 5 | Extension          | 0.994    | 0.994     | 0.994  | 0.994    |

## Modelling

```
import pickle
pickle.dump(cv, open("data1_bin.pickle", "wb"))
```

```
import joblib
filename = 'model_data1_bin.sav'
joblib.dump(model, filename)
```

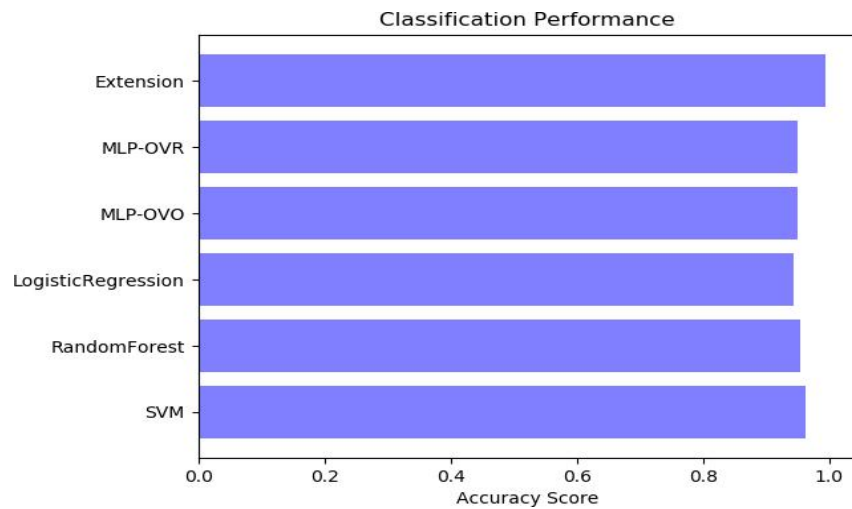
```
['model_data1_bin.sav']
```

## Graph

```
classifier = ML_Model
y_pos = np.arange(len(classifier))
```

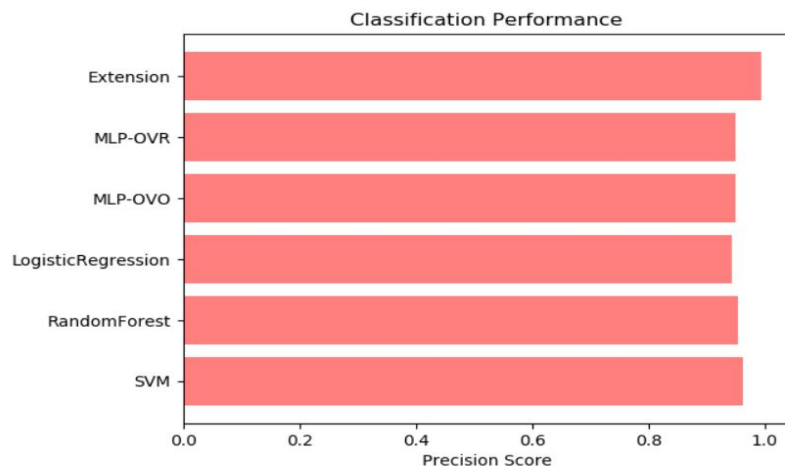
## Accuracy

```
import matplotlib.pyplot as plt2
plt2.barh(y_pos, accuracy, align='center', alpha=0.5,color='blue')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Accuracy Score')
plt2.title('Classification Performance')
plt2.show()
```



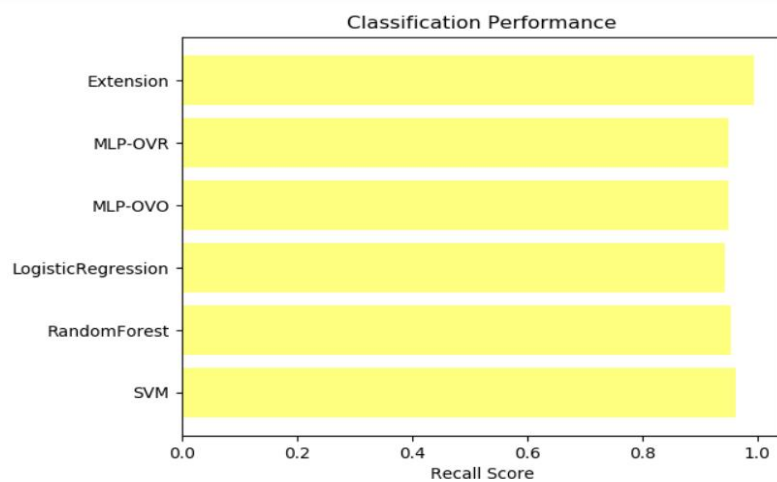
## Precision

```
plt2.barh(y_pos, precision, align='center', alpha=0.5, color='red')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Precision Score')
plt2.title('Classification Performance')
plt2.show()
```



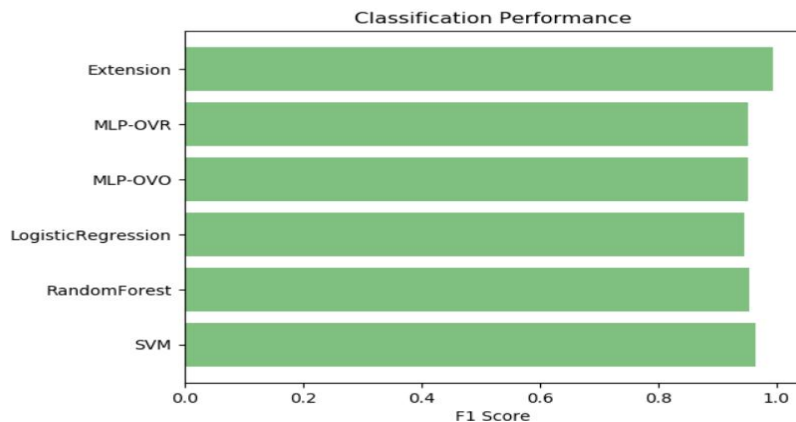
## Recall

```
plt2.barh(y_pos, recall, align='center', alpha=0.5, color='yellow')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Recall Score')
plt2.title('Classification Performance')
plt2.show()
```



## F1 Score

```
plt2.barh(y_pos, f1score, align='center', alpha=0.5,color='green')
plt2.yticks(y_pos, classifier)
plt2.xlabel('F1 Score')
plt2.title('Classification Performance')
plt2.show()
```



## Multi-Class

Collapse 9 child cells under Multi-Class (Press <Shift> to also collapse sibling sections)

```
data = pd.read_csv('dataset1/cyberbullying_tweets.csv')
```

```
data['message'] = data['tweet_text']
```

```
Tweet = []
Labels = []
```

```
for row in data["message"]:
    #tokenize words
    words = word_tokenize(row)
    #remove punctuations
    clean_words = [word.lower() for word in words if word not in set(string.punctuation)]
    #remove stop words
    english_stops = set(stopwords.words('english'))
    characters_to_remove = ["'", "''", "rt", "https", ":", "(", ")", "\u200b", "--", "n't", "'s", "...", "/t.c" ]
    clean_words = [word for word in clean_words if word not in english_stops]
    clean_words = [word for word in clean_words if word not in set(characters_to_remove)]
    #Lematise words
    wordnet_lemmatizer = WordNetLemmatizer()
    lemma_list = [wordnet_lemmatizer.lemmatize(word) for word in clean_words]
    Tweet.append(lemma_list)
```

```
# Import label encoder
from sklearn import preprocessing

# label_encoder object knows
# how to understand word labels.
label_encoder = preprocessing.LabelEncoder()

# Encode labels in column 'species'.
data['cyberbullying_type'] = label_encoder.fit_transform(data['cyberbullying_type'])

data['cyberbullying_type'].unique()
```

```
array([5, 0, 2, 1, 3, 4])
```

```
X = data['message']
y = data['cyberbullying_type']
```

```
# Extract Feature With CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
cv = TfidfVectorizer(stop_words='english')
X = cv.fit_transform(X) # Fit the Data
```

```
smt = SMOTE()
X, y = smt.fit_resample(X, y)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

```

▶ ML_Model = []
accuracy = []
precision = []
recall = []
f1score = []

#function to call for storing the results
def storeResults(model, a,b,c,d):
    ML_Model.append(model)
    accuracy.append(round(a, 3))
    precision.append(round(d, 3))
    recall.append(round(c, 3))
    f1score.append(round(b, 3))

```

## SVM

```

svc = svm.LinearSVC()
svc.fit(X_train, y_train)

y_pred = svc.predict(X_test)

svc_acc = accuracy_score(y_pred, y_test)
svc_prec = precision_score(y_pred, y_test,average='weighted')
svc_rec = recall_score(y_pred, y_test,average='weighted')
svc_f1 = f1_score(y_pred, y_test,average='weighted')

storeResults('SVM',svc_acc,svc_prec,svc_rec,svc_f1)

```

## Random Forest

```

rf = RandomForestClassifier()

rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

rf_acc = accuracy_score(y_pred, y_test)
rf_prec = precision_score(y_pred, y_test,average='weighted')
rf_rec = recall_score(y_pred, y_test,average='weighted')
rf_f1 = f1_score(y_pred, y_test,average='weighted')

storeResults('RandomForest',rf_acc,rf_prec,rf_rec,rf_f1)

```

## LogisticRegression

```

lr = LogisticRegression(multi_class = 'multinomial', solver = 'lbfgs')
lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)

lr_acc = accuracy_score(y_pred, y_test)
lr_prec = precision_score(y_pred, y_test,average='weighted')
lr_rec = recall_score(y_pred, y_test,average='weighted')
lr_f1 = f1_score(y_pred, y_test,average='weighted')

storeResults('LogisticRegression',lr_acc,lr_prec,lr_rec,lr_f1)

```

## MLP - OVO

```
mlp = OneVsOneClassifier(MLPClassifier(random_state=1))  
mlp.fit(X_train, y_train)  
y_pred = mlp.predict(X_test)  
mlp_acc = accuracy_score(y_pred, y_test)  
mlp_prec = precision_score(y_pred, y_test, average='weighted')  
mlp_rec = recall_score(y_pred, y_test, average='weighted')  
mlp_f1 = f1_score(y_pred, y_test, average='weighted')
```

```
storeResults('MLP-OVO', mlp_acc, mlp_prec, mlp_rec, mlp_f1)
```

## MLP - OVR

```
mlp = OneVsRestClassifier(MLPClassifier(random_state=1))  
mlp.fit(X_train, y_train)  
y_pred = mlp.predict(X_test)  
mlp1_acc = accuracy_score(y_pred, y_test)  
mlp1_prec = precision_score(y_pred, y_test, average='weighted')  
mlp1_rec = recall_score(y_pred, y_test, average='weighted')  
mlp1_f1 = f1_score(y_pred, y_test, average='weighted')
```

```
storeResults('MLP-OVR', mlp1_acc, mlp1_prec, mlp1_rec, mlp1_f1)
```

## Extension

```
model = VotingClassifier(estimators= [('BoostDT', bdt), ('BagRF', brf)], voting='soft')  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)  
ext_acc = accuracy_score(y_pred, y_test)  
ext_prec = precision_score(y_pred, y_test, average='weighted')  
ext_rec = recall_score(y_pred, y_test, average='weighted')  
ext_f1 = f1_score(y_pred, y_test, average='weighted')
```

```
storeResults('Extension', ext_acc, ext_prec, ext_rec, ext_f1)
```

## Comparison

```
#creating dataframe  
result = pd.DataFrame({ 'ML Model' : ML_Model,  
                        'Accuracy' : accuracy,  
                        'Precision': precision,  
                        'Recall'   : recall,  
                        'F1_score' : f1score  
                        })
```

result

|   | ML Model           | Accuracy | Precision | Recall | F1_score |
|---|--------------------|----------|-----------|--------|----------|
| 0 | SVM                | 0.832    | 0.830     | 0.832  | 0.830    |
| 1 | RandomForest       | 0.825    | 0.827     | 0.825  | 0.847    |
| 2 | LogisticRegression | 0.820    | 0.815     | 0.820  | 0.818    |
| 3 | MLP-OVO            | 0.322    | 0.458     | 0.322  | 0.957    |
| 4 | MLP-OVR            | 0.634    | 0.631     | 0.634  | 0.705    |
| 5 | Extension          | 0.985    | 0.985     | 0.985  | 0.985    |

## Modelling

```
import pickle
pickle.dump(cv, open("data1_multi.pickle", "wb"))
```

```
import joblib
filename = 'model_data1_multi.sav'
joblib.dump(model, filename)
```

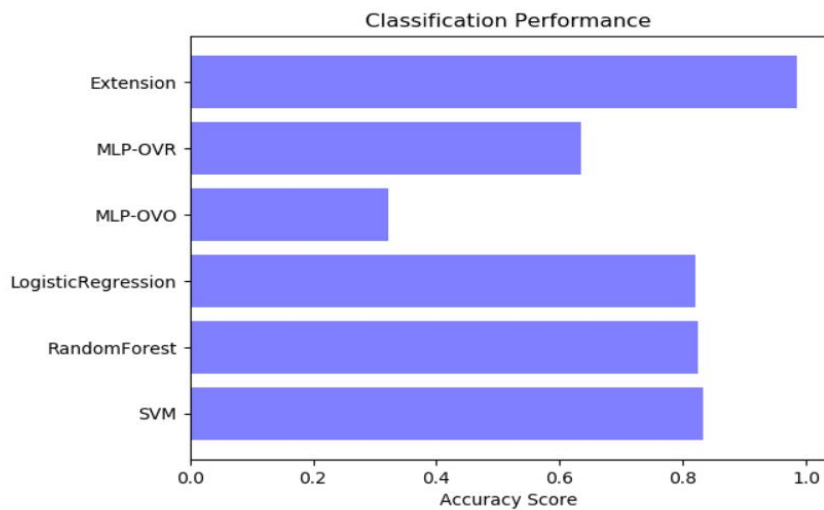
```
['model_data1_multi.sav']
```

## Graph

```
classifier = ML_Model
y_pos = np.arange(len(classifier))
```

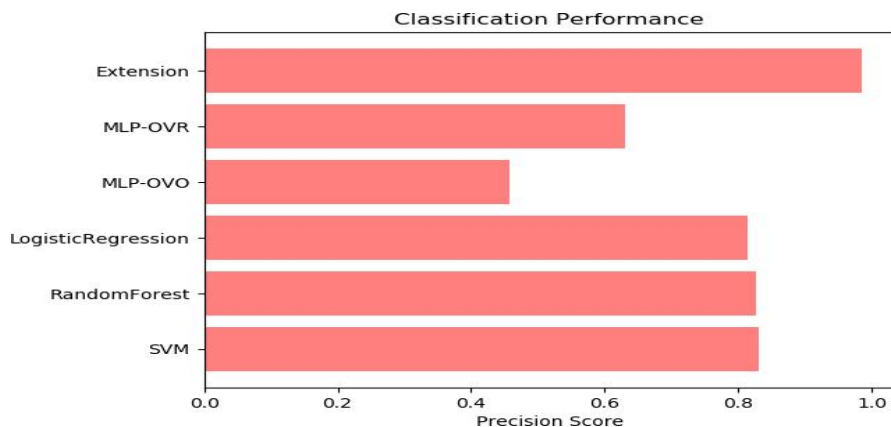
## Accuracy

```
plt2.barh(y_pos, accuracy, align='center', alpha=0.5,color='blue')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Accuracy Score')
plt2.title('Classification Performance')
plt2.show()
```



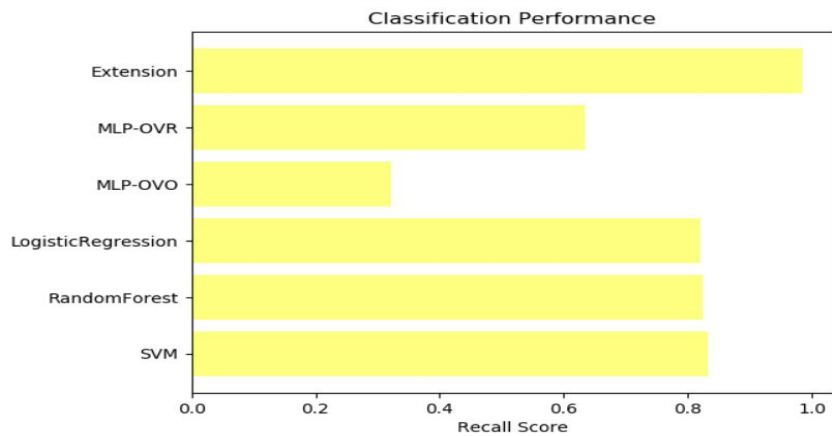
## Precision

```
plt2.barh(y_pos, precision, align='center', alpha=0.5,color='red')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Precision Score')
plt2.title('Classification Performance')
plt2.show()
```



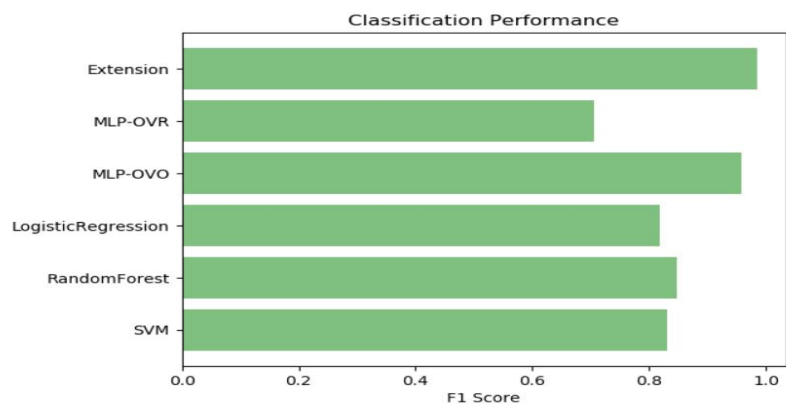
## Recall

```
plt2.barh(y_pos, recall, align='center', alpha=0.5,color='yellow')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Recall Score')
plt2.title('Classification Performance')
plt2.show()
```



## F1 Score

```
plt2.barh(y_pos, f1score, align='center', alpha=0.5,color='green')
plt2.yticks(y_pos, classifier)
plt2.xlabel('F1 Score')
plt2.title('Classification Performance')
plt2.show()
```



## Dataset2.ipynb:

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

data = pd.read_csv('dataset2/cb_multi_labeled_balanced.csv')

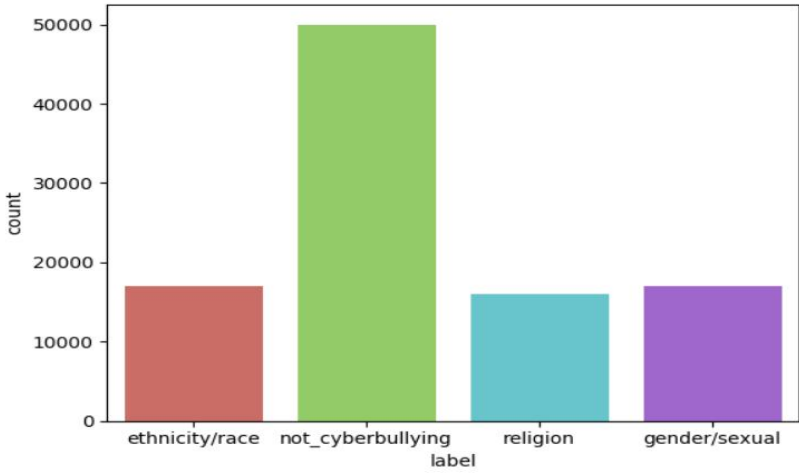
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99990 entries, 0 to 99989
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   text        99990 non-null   object
 1   label       99990 non-null   object
dtypes: object(2)
memory usage: 1.5 MB

data.head()

      text      label
0  @ZubearSays Any real nigga isn't letting this ...  ethnicity/race
1  @MoradoSkittle @prolifejewess @DAConsult @Kell...  not_cyberbullying
2  the only thing i wish, i wish a nigga would      ethnicity/race
3  You saudias are not friends of Muslim idiots c...    religion
4  @JaydenT2399 @TractorLaw @holmes_gael @erconge...    religion

sns.countplot(x='label',data=data, palette='hls')
plt.show()
#plt.savefig('count_plot') mal: the nodule malignancy, 0: benign, 1: malignant
```



```
data['message'] = data['text']

from wordcloud import WordCloud, STOPWORDS
comment_words = ''
stopwords = set(STOPWORDS)

for val in data.message:

    # typecaste each val to string
    val = str(val)

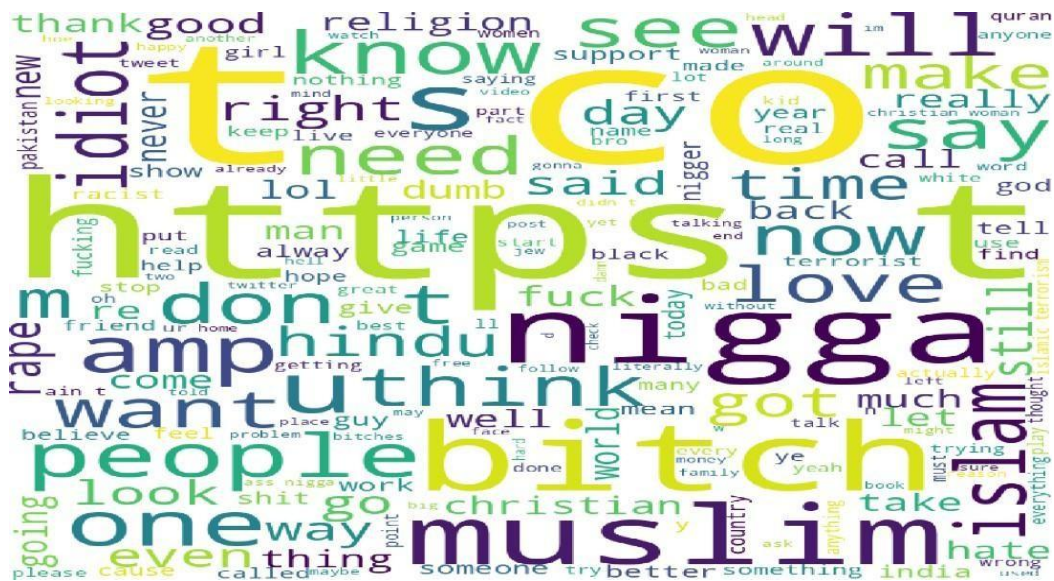
    # split the value
    tokens = val.split()

    # Converts each token into lowercase
    for i in range(len(tokens)):
        tokens[i] = tokens[i].lower()

    comment_words += " ".join(tokens)+" "
```

```
# plot the wordCloud image
plt.figure(figsize = (12, 12), facecolor = None)
plt.imshow(wordcloud)
plt.axis("off")
plt.tight_layout(pad = 0)

plt.show()
```



```
data['label'].value_counts()
```

```
not_cyberbullying      50000
ethnicity/race          17000
gender/sexual           17000
religion                15990
Name: label, dtype: int64
```

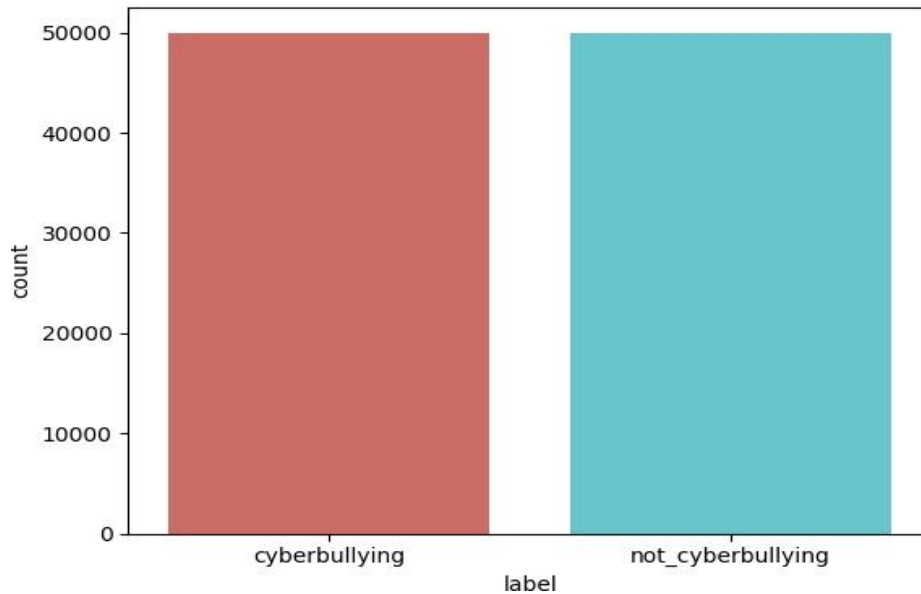
```
def change_label(df):
    df['label'].replace(['religion', 'ethnicity/race', 'gender/sexual'], 'cyberbullying', inplace=True)
```

```
change label(data)
```

```
data['label'].value_counts()
```

```
not_cyberbullying    50000
cyberbullying        49990
Name: label, dtype: int64
```

```
sns.countplot(x='label',data=data, palette='hls')
plt.show()
```



```

Tweet = []
Labels = []

for row in data["message"]:
    #tokenize words
    words = word_tokenize(row)
    #remove punctuations
    clean_words = [word.lower() for word in words if word not in set(string.punctuation)]
    #remove stop words
    english_stops = set(stopwords.words('english'))
    characters_to_remove = ["'", '"', "rt", "https", ":", "(", ")", "\u200b", "--", "n't", "'s", "...", "//t.c" ]
    clean_words = [word for word in clean_words if word not in english_stops]
    clean_words = [word for word in clean_words if word not in set(characters_to_remove)]
    #Lematise words
    wordnet_lemmatizer = WordNetLemmatizer()
    lemma_list = [wordnet_lemmatizer.lemmatize(word) for word in clean_words]
    Tweet.append(lemma_list)

```

```

# Import label encoder
from sklearn import preprocessing

# label_encoder object knows
# how to understand word labels.
label_encoder = preprocessing.LabelEncoder()

# Encode labels in column 'species'.
data['label'] = label_encoder.fit_transform(data['label'])

data['label'].unique()

array([0, 1])

```

```

X = data['message']
y = data['label']

```

```
# Extract Feature With CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
cv = TfidfVectorizer(stop_words='english')
X = cv.fit_transform(X) # Fit the Data
```

```
from imblearn.over_sampling import SMOTE
```

```
smt = SMOTE()
X, y = smt.fit_resample(X, y)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

```
ML_Model = []
accuracy = []
precision = []
recall = []
f1score = []

#function to call for storing the results
def storeResults(model, a,b,c,d):
    ML_Model.append(model)
    accuracy.append(round(a, 3))
    precision.append(round(d, 3))
    recall.append(round(c, 3))
    f1score.append(round(b, 3))
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

## SVM

```
from sklearn import svm
svc = svm.LinearSVC()
svc.fit(X_train, y_train)

y_pred = svc.predict(X_test)

svc_acc = accuracy_score(y_pred, y_test)
svc_prec = precision_score(y_pred, y_test,average='weighted')
svc_rec = recall_score(y_pred, y_test,average='weighted')
svc_f1 = f1_score(y_pred, y_test,average='weighted')

storeResults('SVM',svc_acc,svc_prec,svc_rec,svc_f1)
```

## Random Forest

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier()

rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

rf_acc = accuracy_score(y_pred, y_test)
rf_prec = precision_score(y_pred, y_test, average='weighted')
rf_rec = recall_score(y_pred, y_test, average='weighted')
rf_f1 = f1_score(y_pred, y_test, average='weighted')

storeResults('RandomForest', rf_acc, rf_prec, rf_rec, rf_f1)
```

## Logistic Regression

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(multi_class = 'multinomial', solver = 'lbfgs')
lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)

lr_acc = accuracy_score(y_pred, y_test)
lr_prec = precision_score(y_pred, y_test, average='weighted')
lr_rec = recall_score(y_pred, y_test, average='weighted')
lr_f1 = f1_score(y_pred, y_test, average='weighted')

storeResults('LogisticRegression', lr_acc, lr_prec, lr_rec, lr_f1)
```

## MLP – OVO

```
from sklearn.multiclass import OneVsOneClassifier
from sklearn.neural_network import MLPClassifier

mlp = OneVsOneClassifier(MLPClassifier(random_state=1))

mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)

mlp_acc = accuracy_score(y_pred, y_test)
mlp_prec = precision_score(y_pred, y_test, average='weighted')
mlp_rec = recall_score(y_pred, y_test, average='weighted')
mlp_f1 = f1_score(y_pred, y_test, average='weighted')

storeResults('MLP-OVO', mlp_acc, mlp_prec, mlp_rec, mlp_f1)
```

## MLP – OVR

```
from sklearn.multiclass import OneVsRestClassifier
mlp = OneVsRestClassifier(MLPClassifier(random_state=1))

mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)

mlp1_acc = accuracy_score(y_pred, y_test)
mlp1_prec = precision_score(y_pred, y_test, average='weighted')
mlp1_rec = recall_score(y_pred, y_test, average='weighted')
mlp1_f1 = f1_score(y_pred, y_test, average='weighted')

storeResults('MLP-OVR', mlp1_acc, mlp1_prec, mlp1_rec, mlp1_f1)
```

## Extension

```
from sklearn.ensemble import VotingClassifier, BaggingClassifier, ExtraTreesClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier

brf = BaggingClassifier(RandomForestClassifier())

bdt = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1))

model = VotingClassifier(estimators=[('BoostDT', bdt), ('BagRF', brf)], voting='soft')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

ext_acc = accuracy_score(y_pred, y_test)
ext_prec = precision_score(y_pred, y_test, average='weighted')
ext_rec = recall_score(y_pred, y_test, average='weighted')
ext_f1 = f1_score(y_pred, y_test, average='weighted')

storeResults('Extension', ext_acc, ext_prec, ext_rec, ext_f1)
```

## Comparison

```
#creating dataframe
result = pd.DataFrame({ 'ML_Model' : ML_Model,
                        'Accuracy' : accuracy,
                        'Precision': precision,
                        'Recall'   : recall,
                        'F1_score' : f1score
                        })
```

result

|   | ML_Model           | Accuracy | Precision | Recall | F1_score |
|---|--------------------|----------|-----------|--------|----------|
| 0 | SVM                | 0.988    | 0.987     | 0.988  | 0.988    |
| 1 | RandomForest       | 0.993    | 0.993     | 0.993  | 0.993    |
| 2 | LogisticRegression | 0.978    | 0.978     | 0.978  | 0.979    |
| 3 | MLP-OVO            | 0.489    | 0.657     | 0.489  | 1.000    |
| 4 | MLP-OVR            | 0.968    | 0.968     | 0.968  | 0.970    |
| 5 | Extension          | 1.000    | 1.000     | 1.000  | 1.000    |

## Modelling

```
import pickle
pickle.dump(cv, open("data2_bin.pickle", "wb"))
```

```
import joblib
filename = 'model_data2_bin.sav'
joblib.dump(model, filename)
```

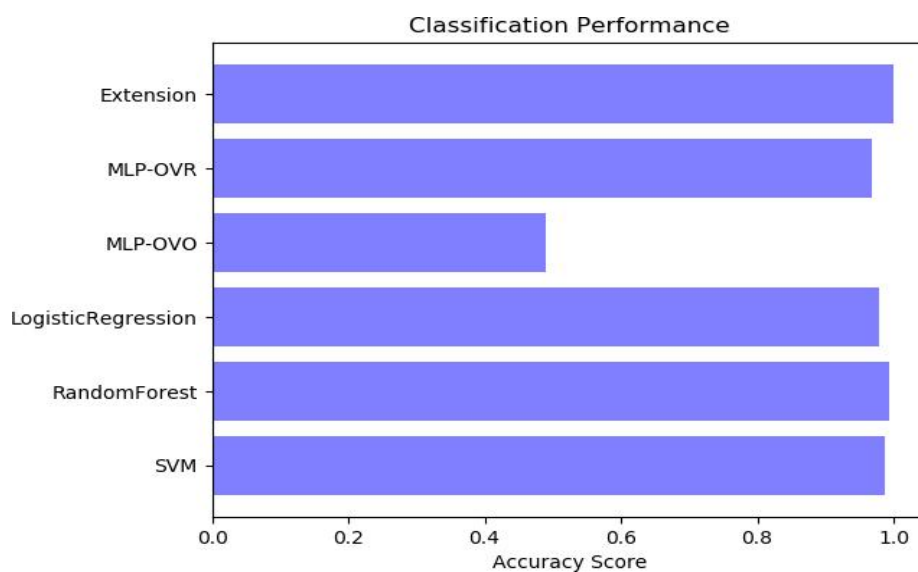
['model\_data2\_bin.sav']

## Graph

```
classifier = ML_Model
y_pos = np.arange(len(classifier))
```

## Accuracy

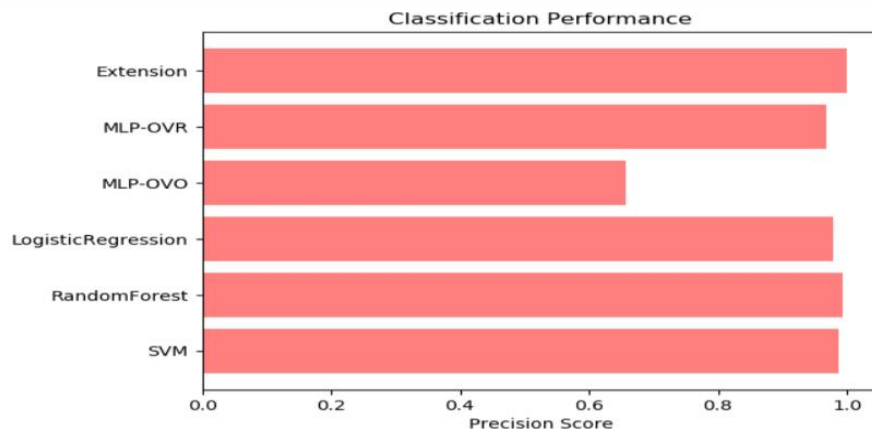
```
import matplotlib.pyplot as plt2
plt2.barh(y_pos, accuracy, align='center', alpha=0.5,color='blue')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Accuracy Score')
plt2.title('Classification Performance')
plt2.show()
```



## Precision

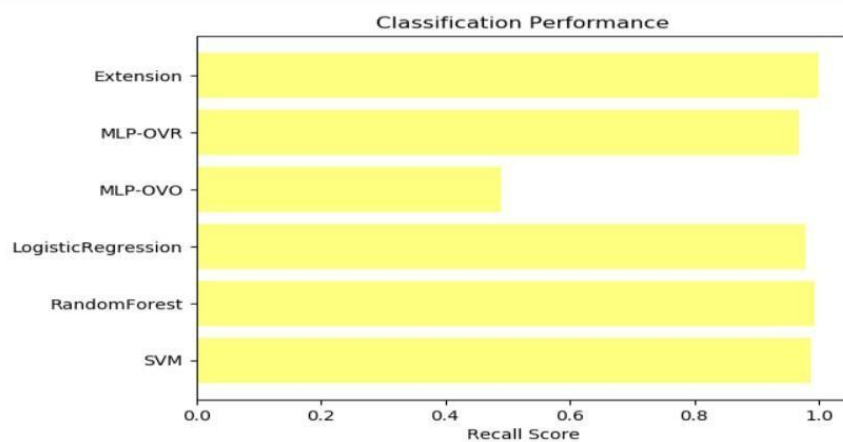
Collapse 1 child cell under Precision (Press <Shift> to also collapse sibling sections)

```
plt2.barh(y_pos, precision, align='center', alpha=0.5,color='red')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Precision Score')
plt2.title('Classification Performance')
plt2.show()
```



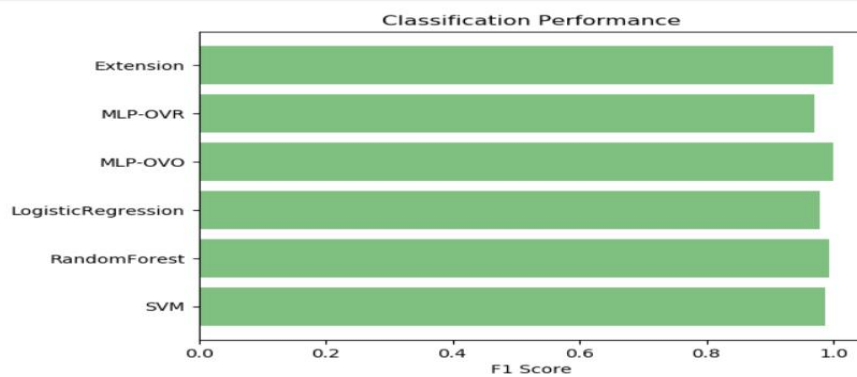
## Recall

```
plt2.barh(y_pos, recall, align='center', alpha=0.5,color='yellow')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Recall Score')
plt2.title('Classification Performance')
plt2.show()
```



## F1 Score

```
plt2.barh(y_pos, f1score, align='center', alpha=0.5,color='green')
plt2.yticks(y_pos, classifier)
plt2.xlabel('F1 Score')
plt2.title('Classification Performance')
plt2.show()
```



## Multi-Class



```
data = pd.read_csv('dataset2/cb_multi_labeled_balanced.csv')
```

```
data['message'] = data['text']
```

```
#print(data.info())
data = pd.concat([
    data[data.label == 'ethnicity/race'].sample(n=10_00),
    data[data.label == 'gender/sexual'].sample(n=10_00),
    data[data.label == 'religion'].sample(n=10_00),
    data[data.label == 'not_cyberbullying'].sample(n=10_00)
])
```

```
Tweet = []
Labels = []
```

```
for row in data["message"]:
    #tokenize words
    words = word_tokenize(row)
    #remove punctuations
    clean_words = [word.lower() for word in words if word not in set(string.punctuation)]
    #remove stop words
    english_stops = set(stopwords.words('english'))
    characters_to_remove = ["'", '`', "rt", "https", ":", "(", ")", "\u200b", "--", "n't", "'s", "...", "//t.c" ]
    clean_words = [word for word in clean_words if word not in english_stops]
    clean_words = [word for word in clean_words if word not in set(characters_to_remove)]
    #Lematise words
    wordnet_lemmatizer = WordNetLemmatizer()
    lemma_list = [wordnet_lemmatizer.lemmatize(word) for word in clean_words]
    Tweet.append(lemma_list)
```

[Code](#)[1](#)

```
# Import label encoder
from sklearn import preprocessing

# label_encoder object knows
# how to understand word labels.
label_encoder = preprocessing.LabelEncoder()

# Encode labels in column 'species'.
data['label'] = label_encoder.fit_transform(data['label'])

data['label'].unique()
```

```
... array([0, 1, 3, 2])
```

```
X = data['message']
y = data['label']
```

```
# Extract Feature With CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
cv = TfidfVectorizer(stop_words='english')
X = cv.fit_transform(X) # Fit the Data
```

```
smt = SMOTE()
X, y = smt.fit_resample(X, y)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

```

ML_Model = []
accuracy = []
precision = []
recall = []
f1score = []

#function to call for storing the results
def storeResults(model, a,b,c,d):
    ML_Model.append(model)
    accuracy.append(round(a, 3))
    precision.append(round(d, 3))
    recall.append(round(c, 3))
    f1score.append(round(b, 3))

```

## SVM

```

svc = svm.LinearSVC()
svc.fit(X_train, y_train)

y_pred = svc.predict(X_test)

svc_acc = accuracy_score(y_pred, y_test)
svc_prec = precision_score(y_pred, y_test,average='weighted')
svc_rec = recall_score(y_pred, y_test,average='weighted')
svc_f1 = f1_score(y_pred, y_test,average='weighted')

storeResults('SVM',svc_acc,svc_prec,svc_rec,svc_f1)

```

## Random Forest

```

rf = RandomForestClassifier()

rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

rf_acc = accuracy_score(y_pred, y_test)
rf_prec = precision_score(y_pred, y_test,average='weighted')
rf_rec = recall_score(y_pred, y_test,average='weighted')
rf_f1 = f1_score(y_pred, y_test,average='weighted')

storeResults('RandomForest',rf_acc,rf_prec,rf_rec,rf_f1)

```

## LogisticRegression

```

lr = LogisticRegression(multi_class = 'multinomial', solver = 'lbfgs')
lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)

lr_acc = accuracy_score(y_pred, y_test)
lr_prec = precision_score(y_pred, y_test,average='weighted')
lr_rec = recall_score(y_pred, y_test,average='weighted')
lr_f1 = f1_score(y_pred, y_test,average='weighted')

storeResults('LogisticRegression',lr_acc,lr_prec,lr_rec,lr_f1)

```

## MLP - OVO

---

```
mlp = OneVsOneClassifier(MLPClassifier(random_state=1))

mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)

mlp_acc = accuracy_score(y_pred, y_test)
mlp_prec = precision_score(y_pred, y_test, average='weighted')
mlp_rec = recall_score(y_pred, y_test, average='weighted')
mlp_f1 = f1_score(y_pred, y_test, average='weighted')
```

---

```
storeResults('MLP-OVO', mlp_acc, mlp_prec, mlp_rec, mlp_f1)
```

---

## MLP - OVR

---

```
mlp = OneVsRestClassifier(MLPClassifier(random_state=1))

mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)

mlp1_acc = accuracy_score(y_pred, y_test)
mlp1_prec = precision_score(y_pred, y_test, average='weighted')
mlp1_rec = recall_score(y_pred, y_test, average='weighted')
mlp1_f1 = f1_score(y_pred, y_test, average='weighted')
```

---

```
storeResults('MLP-OVR', mlp1_acc, mlp1_prec, mlp1_rec, mlp1_f1)
```

---

## Extension

---

```
model = VotingClassifier(estimators= [('BoostDT', bdt), ('BagRF', brf)], voting='soft')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

ext_acc = accuracy_score(y_pred, y_test)
ext_prec = precision_score(y_pred, y_test, average='weighted')
ext_rec = recall_score(y_pred, y_test, average='weighted')
ext_f1 = f1_score(y_pred, y_test, average='weighted')
```

---

```
storeResults('Extension', ext_acc, ext_prec, ext_rec, ext_f1)
```

---

## Comparison

---

```
#creating dataframe
result = pd.DataFrame({ 'ML_Model' : ML_Model,
                        'Accuracy' : accuracy,
                        'Precision': precision,
                        'Recall'   : recall,
                        'F1_score' : f1score
                        })
```

---

result

|   | ML Model           | Accuracy | Precision | Recall | F1_score |
|---|--------------------|----------|-----------|--------|----------|
| 0 | SVM                | 0.976    | 0.976     | 0.976  | 0.977    |
| 1 | RandomForest       | 0.981    | 0.981     | 0.981  | 0.981    |
| 2 | LogisticRegression | 0.935    | 0.934     | 0.935  | 0.940    |
| 3 | MLP-OVO            | 0.532    | 0.607     | 0.532  | 0.721    |
| 4 | MLP-OVR            | 0.715    | 0.723     | 0.715  | 0.765    |
| 5 | Extension          | 0.997    | 0.997     | 0.997  | 0.997    |

## Modelling

```
import pickle
pickle.dump(cv, open("data2_multi.pickle", "wb"))
```

```
import joblib
filename = 'model_data2_multi.sav'
joblib.dump(model, filename)
```

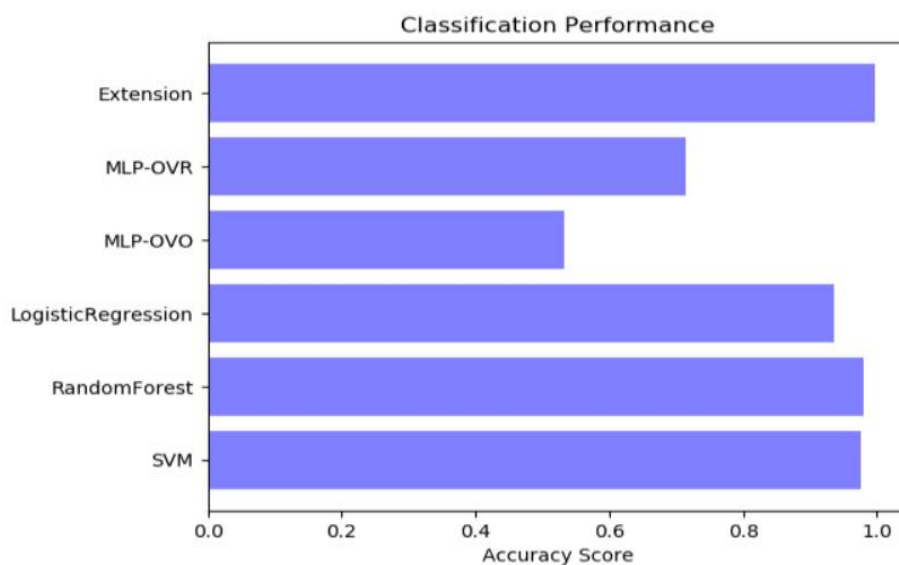
```
['model_data2_multi.sav']
```

## Graph

```
classifier = ML_Model
y_pos = np.arange(len(classifier))
```

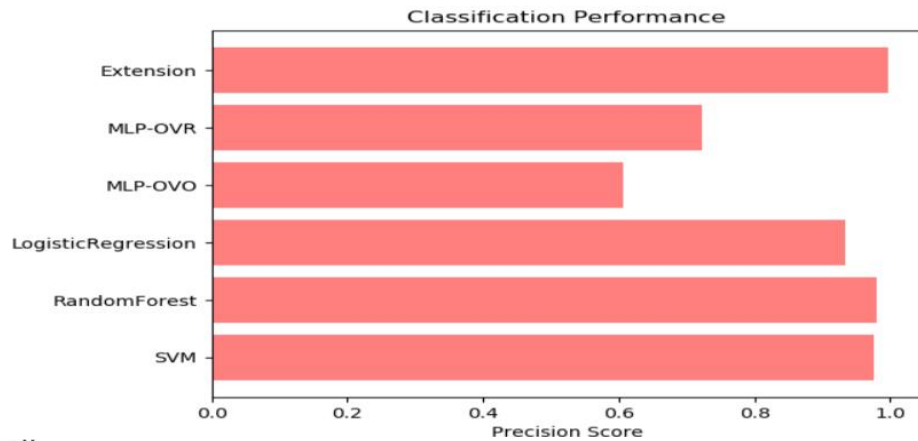
## Accuracy

```
plt2.barh(y_pos, accuracy, align='center', alpha=0.5,color='blue')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Accuracy Score')
plt2.title('Classification Performance')
plt2.show()
```



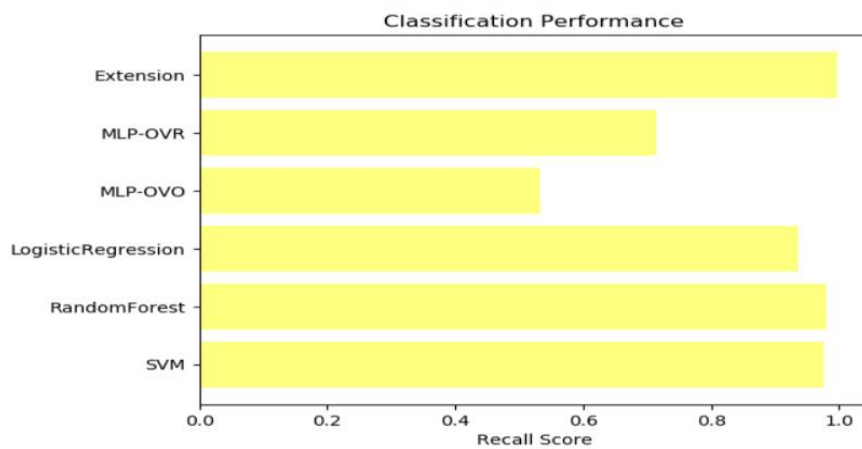
## Precision

```
plt2.barh(y_pos, precision, align='center', alpha=0.5,color='red')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Precision Score')
plt2.title('Classification Performance')
plt2.show()
```



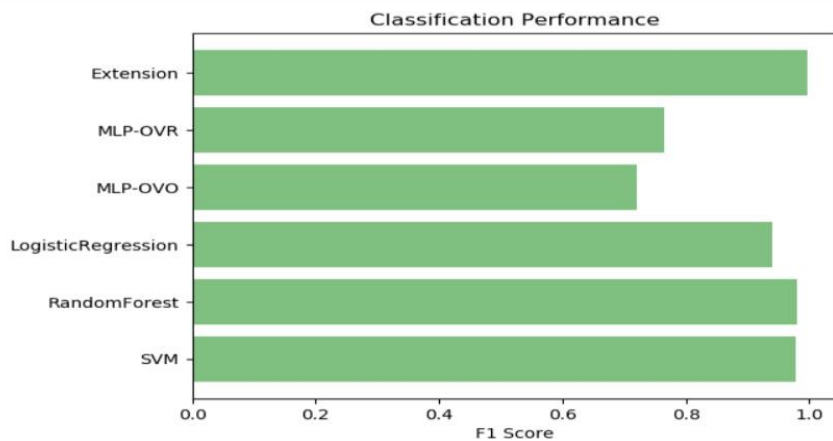
## Recall

```
plt2.barh(y_pos, recall, align='center', alpha=0.5,color='yellow')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Recall Score')
plt2.title('Classification Performance')
plt2.show()
```



## F1 Score

```
plt2.barh(y_pos, f1score, align='center', alpha=0.5,color='green')
plt2.yticks(y_pos, classifier)
plt2.xlabel('F1 Score')
plt2.title('Classification Performance')
plt2.show()
```



## Backend:

### Views.py

Backend > views.py

```
1  from django.shortcuts import render, redirect
2  from django.contrib.auth.models import User
3  from django.contrib.auth import authenticate, login, logout
4  from django.contrib import messages
5
6  def index(request):
7      return render(request, "index.html")
8
9  def login_page(request):
10     return render(request, "login.html")
11
12  def register_page(request):
13     return render(request, "register.html")
14
15     # Define the login function
16  def user_login(request):
17     if request.method == "POST":
18         username = request.POST.get('username')
19         password = request.POST.get('password')
20
21         # Authenticate user
22         user = authenticate(request, username=username, password=password)
23
24         if user is not None:
25             if not user.is_active:
26                 # User is inactive
27                 messages.error(request, "Your account is inactive. Please contact the admin.")
28                 return redirect('login_page')
29
30                 # Login the user
31                 login(request, user)
32
33             if user.is_staff or user.is_superuser:
34                 # Redirect to admin home if user is staff
35                 return redirect('adminhome')
36             else:
37                 # Redirect to user home if user is not staff
38                 return redirect('userhome')
```

```

39         else:
40             # Invalid username or password
41             messages.error(request, "Invalid username or password.")
42             return redirect('login_page')
43
44     return render(request, 'login.html')
45
46 # Define the user registration function
47 def user_registration(request):
48     if request.method == "POST":
49         username = request.POST.get('username')
50         email = request.POST.get('email')
51         password = request.POST.get('password')
52         confirm_password = request.POST.get('confirm_password')
53         first_name = request.POST.get('first_name')
54         last_name = request.POST.get('last_name')
55
56         # Check if passwords match
57         if password != confirm_password:
58             messages.error(request, "Passwords do not match.")
59             return redirect('register_page')
60
61         # Check if username already exists
62         if User.objects.filter(username=username).exists():
63             messages.error(request, "Username already exists.")
64             return redirect('register_page')
65
66         # Check if email already exists
67         if User.objects.filter(email=email).exists():
68             messages.error(request, "Email already exists.")
69             return redirect('register_page')
70
71         # Create the user with is_active set to False
72         user = User.objects.create_user(
73             username=username,
74             email=email,
75             password=password,
76             first_name=first_name,
77             last_name=last_name
78         )
79         user.is_active = False # Set is_active to False by default
80         user.save()
81
82         messages.success(request, "Registration successful! Please wait for admin approval.")
83         return redirect('login_page')
84
85     return render(request, 'register.html')
86
87 # Define the logout function
88 def user_logout(request):
89     logout(request)
90     messages.success(request, "You have been logged out successfully.")
91     return redirect('login page')

```

## Views.py

User > views.py > user\_prediction1

```
1  from django.shortcuts import render
2  from .models import UserPrediction
3  import joblib
4  import pickle
5  import string
6  import nltk
7  import os
8  from django.conf import settings
9
10 from django.shortcuts import render
11 from nltk.corpus import stopwords
12 from nltk.tokenize import word_tokenize
13 from nltk.stem import WordNetLemmatizer
14 import speech recognition as sr
15
16 nltk.download('punkt')
17 nltk.download('wordnet')
18 nltk.download('stopwords')
19
20 model = joblib.load("model/model_data1_bin.sav")
21 with open("model/data1_bin.pickle", "rb") as f:
22     vectorizer = pickle.load(f)
23
24 model2 = joblib.load("model/model_data2_bin.sav")
25 with open("model/data2_bin.pickle", "rb") as f:
26     vectorizer2 = pickle.load(f)
27
28 def preprocess_text(text):
29     words = word_tokenize(text)
30     clean_words = [word.lower() for word in words if word not in string.punctuation]
31
32     stop_words = set(stopwords.words('english'))
33     extra_removals = ["'", "''", "rt", "https", "[]", "()", "!", ":", "\u200b", "--", "n't", "'s", "...", "//t.c"]
34
35     clean_words = [word for word in clean_words if word not in stop_words and word not in extra_removals]
36
37     lemmatizer = WordNetLemmatizer()
38     lemmatized = [lemmatizer.lemmatize(word) for word in clean_words]
39
40     return " ".join(lemmatized)
41
42 def userhome(request):
43     user = request.user
44     return render(request, 'User/userhome.html', {'user':user})
45
46
47 def user_prediction1(request):
48     prediction_label = None
```

```

49     input_text = None
50
51     if request.method == "POST":
52         input_text = request.POST.get("user_input", "")
53         processed_text = preprocess_text(input_text)
54         vectorized_input = vectorizer.transform([processed_text])
55         prediction = model.predict(vectorized_input)
56         prediction_label = "Not Cyberbullying" if prediction[0] == 0 else "Cyberbullying"
57
58         UserPrediction.objects.create(
59             input_text=input_text,
60             prediction_result=prediction_label
61         )
62
63     return render(request, 'User/userprediction1.html', {
64         'input_text': input_text,
65         'prediction_label': prediction_label
66     })
67
68 def user_prediction2(request):
69     prediction_label = None
70     input_text = None
71
72     if request.method == "POST":
73         input_text = request.POST.get("user_input", "")
74         processed_text = preprocess_text(input_text)
75         vectorized_input = vectorizer2.transform([processed_text])
76         prediction = model2.predict(vectorized_input)
77         print(prediction)
78         prediction_label = "Not Cyberbullying" if prediction[0] == 1 else "Cyberbullying"
79
80         UserPrediction.objects.create(
81             input_text=input_text,
82             prediction_result=prediction_label
83         )
84
85     return render(request, 'User/userprediction2.html', {
86         'input_text': input_text,
87         'prediction_label': prediction_label
88     })
89
90 def Graphs(request):
91     graphs_folder = os.path.join(settings.BASE_DIR, 'static', 'graphs')
92     static_url = settings.STATIC_URL + 'graphs/'
93

```

```

94     comparison_images = []
95
96     try:
97         for i in range(1, 12):
98             left = f'd1o{i}.png'
99             right = f'd2o{i}.png'
100
101             if left in os.listdir(graphs_folder) and right in os.listdir(graphs_folder):
102                 comparison_images.append({
103                     'left': static_url + left,
104                     'right': static_url + right,
105                     'label': f'Comparison {i}'
106                 })
107         except FileNotFoundError:
108             comparison_images = []
109
110     return render(request, 'User/graphs.html', {'comparisons': comparison_images})
111
112 def uservoice_prediction1(request):
113     prediction_label = None
114     input_text = None
115
116     if request.method == "POST":
117         input_text = request.POST.get("voice_input", "")
118         processed_text = preprocess_text(input_text)
119         vectorized_input = vectorizer2.transform([processed_text])
120         prediction = model2.predict(vectorized_input)
121         prediction_label = "Not Cyberbullying" if prediction[0] == 0 else "Cyberbullying"
122
123     return render(request, 'User/uservoiceprediction1.html', {
124         'input_text': input_text,
125         'prediction_label': prediction_label
126     })

```

## Frontend:

### Base.html:

```
templates > base.html > html > body.index-page > header#header.header.d-flex.align-items-center.light-background.sticky-top > div.container-fluid.position-relative.d-flex.align-items-center.justify-content-between > nav#navmenu.nav
1 <!DOCTYPE html>
2 {% load static%}
3 <html lang="en">
4
5 <head>
6   <meta charset="utf-8">
7   <meta content="width=device-width, initial-scale=1.0" name="viewport">
8   <title>Social Media Forensics</title>
9   <meta name="description" content="">
10  <meta name="keywords" content="">
11
12  <!-- Fonts -->
13  <link href="https://fonts.googleapis.com" rel="preconnect">
14  <link href="https://fonts.gstatic.com" rel="preconnect" crossorigin>
15  <link href="https://fonts.googleapis.com/css2?family=Roboto:ital,wght@0,100;0,300;0,400;0,500;0,700;0,900;1,100;1,300;1,400;1,500;1,700;1,900&family=Poppins:ital,wght@0,100;0,200;0,300;0,400;0,500;0,700;0,800;0,900;1,100;1,200;1,300;1,400;1,500;1,600;1,700;1,800;1,900&family=Redway:ital,wght@0,100;0,200;0,300;0,400;0,500;0,600;0,700;0,800;0,900;1,100;1,200;1,300;1,400;1,500;1,600;1,700;1,800;1,900&display=swap" rel="stylesheet">
16
17  <!-- Vendor CSS Files -->
18  <link href="{% static 'vendor/bootstrap/css/bootstrap.min.css' %}" rel="stylesheet">
19  <link href="{% static 'vendor/bootstrap-icons/bootstrap-icons.css' %}" rel="stylesheet">
20  <link href="{% static 'vendor/aos/aos.css' %}" rel="stylesheet">
21  <link href="{% static 'vendor/swiper/swiper-bundle.min.css' %}" rel="stylesheet">
22  <link href="{% static 'vendor/glightbox/css/glightbox.min.css' %}" rel="stylesheet">
23
24  <!-- Main CSS File -->
25  <link href="{% static 'css/main.css' %}" rel="stylesheet">
26
27
28 </head>
29
30 <body class="index-page">
31
32 <header id="header" class="header d-flex align-items-center light-background sticky-top">
33   <div class="container-fluid position-relative d-flex align-items-center justify-content-between">
34
35     <a class="logo d-flex align-items-center flex-auto flex-shrink-0">
36       <!-- Uncomment the line below if you also wish to use an image logo -->
37       
38       <h1 class="sitename">social media forensics: An Adaptive cyberbullying Related Hate Speech Detection</h1>
39     </a>
40
41     <nav id="navmenu" class="navmenu">
42       <ul>
43         <li><a href="{% url 'home page' %}">Home</a></li>
44         <li><a href="{% url 'login page' %}">Login</a></li>
45       </ul>
46     <div class="mobile-nav-toggle d-flex flex-direction row-reverse"></div>
```

```

47     </nav>
48
49     <div class="header-social-links">
50     </div>
51
52 </div>
53 </header>
54
55 <main class="main">
56
57     {%block contents%}
58
59     {%endblock%}
60
61 </main>
62
63
64 <!-- Scroll Top -->
65 <a href="#" id="scroll-top" class="scroll-top d-flex align-items-center justify-content-center"><i class="bi bi-arrow-up-short"></i></a>
66
67 <!-- Preloader -->
68 <div id="preloader"></div>
69
70 <!-- Vendor JS Files -->
71 <script src="{% static 'vendor/bootstrap/js/bootstrap.bundle.min.js' %}"></script>
72 <script src="{% static 'vendor/php-email-form/validate.js' %}"></script>
73 <script src="{% static 'vendor/aos/aos.js' %}"></script>
74 <script src="{% static 'vendor/waypoints/noframework.waypoints.js' %}"></script>
75 <script src="{% static 'vendor/purecounter/purecounter_vanilla.js' %}"></script>
76 <script src="{% static 'vendor/swiper/swiper-bundle.min.js' %}"></script>
77 <script src="{% static 'vendor/glightbox/js/glightbox.min.js' %}"></script>
78 <script src="{% static 'vendor/imagesloaded/imagesloaded.pkgd.min.js' %}"></script>
79 <script src="{% static 'vendor/isotope-layout/isotope.pkgd.min.js' %}"></script>
80
81 <!-- Main JS File -->
82 <script src="{% static 'js/main.js' %}"></script>
83
84 </body>
85
86 </html>

```

## Main.js

static > js > JS mainjs > ...

```
3 (function() {
4   "use strict";
5
6   /**
7    * Apply .scrolled class to the body as the page is scrolled down
8    */
9   function toggleScrolled() {
10     const selectBody = document.querySelector('body');
11     const selectHeader = document.querySelector('#header');
12     if (!selectHeader.classList.contains('scroll-up-sticky') && !selectHeader.classList.contains('sticky-top') && !selectHeader.classList.contains('fixed-top')) return;
13     window.scrollY > 100 ? selectBody.classList.add('scrolled') : selectBody.classList.remove('scrolled');
14   }
15
16   document.addEventListener('scroll', toggleScrolled);
17   window.addEventListener('load', toggleScrolled);
18
19   /**
20    * Mobile nav toggle
21    */
22   const mobileNavToggleBtn = document.querySelector('.mobile-nav-toggle');
23
24   function mobileNavToggle() {
25     document.querySelector('body').classList.toggle('mobile-nav-active');
26     mobileNavToggleBtn.classList.toggle('bi-list');
27     mobileNavToggleBtn.classList.toggle('bi-x');
28   }
29   mobileNavToggleBtn.addEventListener('click', mobileNavToggle);
30
31   /**
32    * Hide mobile nav on same-page/hash links
33    */
34   document.querySelectorAll('#navmenu a').forEach(navmenu => {
35     navmenu.addEventListener('click', () => {
36       if (document.querySelector('.mobile-nav-active')) {
37         mobileNavToggle();
38       }
39     });
40   });
41
42   /**
43    * Toggle mobile nav dropdowns
44    */
45   document.querySelectorAll('.navmenu .toggle-dropdown').forEach(navmenu => {
46     navmenu.addEventListener('click', function(e) {
47       e.preventDefault();
48       this.parentNode.classList.toggle('active');
49       this.parentNode.nextElementSibling.classList.toggle('dropdown-active');
50     });
51   });
52 }
```

```

51     e.stopImmediatePropagation();
52   });
53 });
54
55 /**
56  * Preloader
57  */
58 const preloader = document.querySelector('#preloader');
59 if (preloader) {
60   window.addEventListener('load', () => {
61     preloader.remove();
62   });
63 }
64
65 /**
66  * Scroll top button
67  */
68 let scrollTop = document.querySelector('.scroll-top');
69
70 function toggleScrollTop() {
71   if (scrollTop) {
72     window.scrollY > 100 ? scrollTop.classList.add('active') : scrollTop.classList.remove('active');
73   }
74 }
75 scrollTop.addEventListener('click', (e) => {
76   e.preventDefault();
77   window.scrollTo({
78     top: 0,
79     behavior: 'smooth'
80   });
81 });
82
83 window.addEventListener('load', toggleScrollTop);
84 document.addEventListener('scroll', toggleScrollTop);
85
86 /**
87  * Animation on scroll function and init
88  */
89 function aosInit() {
90   AOS.init({
91     duration: 600,
92     easing: 'ease-in-out',
93     once: true,
94     mirror: false
95   });

```

Click to add a breakpoint

```

97 window.addEventListener('load', aosInit);
98
99 /**
100  * Animate the skills items on reveal
101  */
102 let skillsAnimation = document.querySelectorAll('.skills-animation');
103 skillsAnimation.forEach((item) => {
104     new Waypoint({
105         element: item,
106         offset: '80%',
107         handler: function(direction) {
108             let progress = item.querySelectorAll('.progress .progress-bar');
109             progress.forEach(el => {
110                 el.style.width = el.getAttribute('aria-valuenow') + '%';
111             });
112         }
113     });
114 });
115
116 /**
117  * Initiate Pure Counter
118  */
119 new PureCounter();
120
121 /**
122  * Init swiper sliders
123  */
124 function initSwiper() {
125     document.querySelectorAll(".init-swiper").forEach(function(swiperElement) {
126         let config = JSON.parse(
127             swiperElement.querySelector(".swiper-config").innerHTML.trim()
128         );
129
130         if (swiperElement.classList.contains("swiper-tab")) {
131             initSwiperWithCustomPagination(swiperElement, config);
132         } else {
133             new Swiper(swiperElement, config);
134         }
135     });
136 }
137
138 window.addEventListener("load", initSwiper);
139
140 /**
141  * Initiate glightbox

```

```

Click to add a breakpoint
143 const glightbox = GLightbox({
144   selector: '.glightbox'
145 });
146
147 /**
148  * Init isotope layout and filters
149  */
150 document.querySelectorAll('.isotope-layout').forEach(function(isotopeItem) {
151   let layout = isotopeItem.getAttribute('data-layout') ?? 'masonry';
152   let filter = isotopeItem.getAttribute('data-default-filter') ?? '*';
153   let sort = isotopeItem.getAttribute('data-sort') ?? 'original-order';
154
155   let initIsotope;
156   imagesLoaded(isotopeItem.querySelector('.isotope-container'), function() {
157     initIsotope = new Isotope(isotopeItem.querySelector('.isotope-container'), {
158       itemSelector: '.isotope-item',
159       layoutMode: layout,
160       filter: filter,
161       sortBy: sort
162     });
163   });
164
165   isotopeItem.querySelectorAll('.isotope-filters li').forEach(function(filters) {
166     filters.addEventListener('click', function() {
167       isotopeItem.querySelector('.isotope-filters .filter-active').classList.remove('filter-active');
168       this.classList.add('filter-active');
169       initIsotope.arrange({
170         filter: this.getAttribute('data-filter')
171       });
172       if (typeof aosInit === 'function') {
173         aosInit();
174       }
175     }, false);
176   });
177
178 });
179
180 })();

```

## Userveprediction1.html

```
1  {% extends 'User/userbase.html' %}
2  {% block contents %}
3  <section class="container py-5">
4  <div class="row justify-content-center">
5  <div class="col-lg-8" data-aos="fade-up">
6  <div class="card p-4 shadow rounded-4">
7      <h2 class="mb-4 text-center">Voice Analysis</h2>
8      <form method="POST">
9          {% csrf_token %}
10         <div class="mb-3">
11             <label class="form-label">Click and speak:</label>
12             <button type="button" class="btn btn-secondary mb-2" onclick="startRecognition()"> Start Voice Input</button>
13             <textarea id="voiceInput" name="voice_input" class="form-control" rows="6" readonly>{{ input_text }}</textarea>
14         </div>
15         <div class="text-center">
16             <button type="submit" class="btn btn-primary px-4 py-2">Predict</button>
17         </div>
18     </form>
19
20     {% if prediction_label %}
21     <div class="alert alert-info mt-4 text-center" role="alert">
22         <h5 class="mb-0">Prediction: {{ prediction_label }}</h5>
23     </div>
24     {% endif %}
25 </div>
26 </div>
27 </div>
28 </section>
29
30 <script>
31 function startRecognition() {
32     const recognition = new (window.SpeechRecognition || window.webkitSpeechRecognition)();
33     recognition.lang = 'en-US';
34     recognition.interimResults = false;
35     recognition.maxAlternatives = 1;
36
37     recognition.start();
38
39     recognition.onresult = function(event) {
40         const transcript = event.results[0][0].transcript;
41         document.getElementById('voiceInput').value = transcript;
42     };
43
44     recognition.onerror = function(event) {
45         alert('Speech recognition error: ' + event.error);
46     };
47 }
48 </script>
49 {% endblock %}
50
```

## Userprediction2.html

templates > User > userprediction2.html > ...

Click to add a breakpoint `er/userbase.html' %}`

```
3  {% block contents %}
4  <section class="container py-5">
5      <div class="row justify-content-center">
6          <div class="col-lg-8" data-aos="fade-up">
7              <div class="card p-4 shadow rounded-4">
8                  <h2 class="mb-4 text-center">Text Analysis</h2>
9                  <form method="POST">
10                     {% csrf_token %}
11                     <div class="mb-3">
12                         <label for="user_input" class="form-label">Enter text:</label>
13                         <textarea name="user_input" class="form-control" rows="6">{{ input_text }}</textarea>
14                     </div>
15                     <div class="text-center">
16                         <button type="submit" class="btn btn-primary px-4 py-2">Predict</button>
17                     </div>
18                 </form>
19
20                 {% if prediction_label %}
21                     <div class="alert alert-info mt-4 text-center" role="alert">
22                         <h5 class="mb-0">Prediction: {{ prediction_label }}</h5>
23                     </div>
24                 {% endif %}
25             </div>
26          </div>
27      </div>
28  </section>
29  {% endblock %}
30
```

## **6.2 Implementation:**

### **6.2.1 Front-End Implementation:**

The front-end of the cyberbullying detection system provides a simple, responsive, and role-based user interface. The main modules include User Registration, User Login, Admin Panel, Text-based Prediction, and Voice-based Prediction.

The Registration and Login modules enable secure user authentication and controlled system access. Input validation is applied to ensure correctness and prevent invalid submissions. The Admin Panel allows authorized administrators to review usage logs, monitor predictions, and manage users and datasets.

The Prediction module enables users to submit textual content for analysis. Once submitted, the input is sent to the backend through REST APIs. The resulting classification is returned and displayed as a clear cyberbullying category. The system intentionally presents only the final category output to keep the interface straightforward and easy to interpret.

The Voice-based Prediction feature allows users to speak instead of typing. Recorded audio is converted to text using a speech-to-text component and the extracted text is then processed through the same classification pipeline. Consistent layouts, clear navigation, and structured feedback enhance usability and accessibility.

### **6.2.2 Backend Implementation (Django):**

The backend is implemented using Django, which provides a secure and scalable framework. The Model–View–Template architecture organizes system functionality into coherent layers:

- Models store user profiles, activity logs, and prediction records.
- Views manage requests, perform validation, and trigger prediction services.
- URLs / APIs define endpoints for authentication, text submission, voice-converted text submission, and result retrieval.

Security mechanisms such as authentication controls, middleware checks, and request validation ensure integrity of data processing. Prediction history is stored to support auditing and further analysis.

### **6.2.3 Model Integration and Processing Workflow**

The machine-learning module is integrated as a backend service within Django. When a text request is received, it passes through the preprocessing pipeline that includes normalization, tokenization, stop- word removal, lemmatization, and TF-IDF feature generation.

Classification is performed using an ensemble approach that combines Boosted Decision Tree (BoostDT) and Bagging Random Forest (BagRF) through Soft Voting. Each model produces a decision, and the voting mechanism determines the final category label returned to the system. SMOTE-balanced datasets and stored model artifacts ensure stable behavior across input variations.

Responses are returned to the front-end in structured JSON format and rendered to the user as the predicted cyberbullying class.

### **6.2.4 Deployment and Reliability**

The system is deployed on a standard server configuration using environment-based settings for security. Static assets are optimized, and REST endpoints are tested for consistent behavior under multiple usage scenarios. Unit and integration tests validate preprocessing, API interaction, and classification response handling.

### **6.2.5 Conclusion**

The implementation successfully integrates a user-friendly interface, a secure Django backend, and an ensemble-based machine-learning framework into a unified cyberbullying detection platform. Both text and voice-based inputs are supported, predictions are generated in real time, and administrative monitoring features enable oversight and maintenance. The modular architecture allows future enhancements such as multilingual support, advanced deep-learning integration, and real-time moderation capabilities. Overall, the implemented system demonstrates a practical, scalable approach to improving online safety through automated cyberbullying detection.

## 7. SYSTEM TESTING

System testing is a critical activity that ensures the developed cyberbullying detection system performs accurately, consistently, and reliably under real operating conditions. The primary purpose of testing is to identify potential defects, validate system behavior, and confirm that all functional and non-functional requirements have been met.

In this project, system testing focuses on validating every major module, including front-end interfaces, Django backend services, preprocessing components, and the ensemble-based classification engine integrating Boosted Decision Tree (BoostDT) and Bagging Random Forest (BagRF) through Soft Voting. Testing verifies that user interactions, dataset handling, model prediction logic, and output presentation operate as expected without failures or inconsistencies.

System testing was performed across multiple scenarios and datasets to ensure correctness, robustness, and usability. Special emphasis was placed on classification behavior, handling of edge-case text inputs, and stability during repeated prediction requests.

### 7.1 Types of System Testing

#### 7.1.1 Unit Testing

Unit testing was carried out to validate individual software components independently. Each Django view, function, preprocessing routine, and classifier interaction module was executed with controlled input values to verify expected outputs.

Key focus areas included:

- tokenization, normalization, and lemmatization behavior
- TF-IDF feature vector generation
- internal logic of ensemble combination
- database operations for login, registration, and predictions

Unit testing ensured that every internal logical path executed correctly and no unexpected conditions occurred. Failures during this stage would have been easier to isolate and correct before integration.

### **7.1.2 Integration Testing**

Integration testing examined whether combined components interacted correctly once they were linked together.

Modules tested in combination included:

- front-end request submission with backend API responses
- preprocessing and feature extraction chained with classification
- ensemble model logic when receiving outputs from BoostDT and BagRF
- prediction logging and storage in the database

This testing stage confirmed that individually correct components functioned properly when executed together as a single workflow. Particular attention was paid to ensuring correct feature alignment between models and stability in Soft Voting combination.

### **7.1.3 Functional Testing**

Functional testing validated that each feature performed according to specification and user expectations. Test cases simulated actual user scenarios such as registration, login, text submission, and voice-based prediction.

Major validation rules included:

- valid inputs are processed successfully
- invalid or empty inputs are rejected gracefully
- correct cyberbullying category is displayed for each prediction
- navigation links and page workflows operate correctly

Functional testing confirmed that system features were clearly available, usable, and responsive.

### **7.1.4 System Testing**

System testing evaluated the project as a complete application. The focus was on overall reliability, consistency of behavior, and the accuracy of outcomes when used in real conditions.

Tests verified:

- coordinated execution across modules
- correct response to large datasets
- classification accuracy under varying abuse patterns

- stability during repeated sequential predictions

The testing demonstrated that the configuration yields predictable and correct results consistent with documented requirements.

### **7.1.5 White-Box Testing**

White-box testing was applied to internal processing components including preprocessing pipelines and ensemble computation logic. Testers observed internal variable flows, code execution branches, and probability aggregation to ensure correct model contributions in Soft Voting.

### **7.1.6 Black-Box Testing**

Black-box testing evaluated the system purely from the user's perspective, without examining internal code. Inputs were submitted through user forms and prediction outputs were observed, ensuring that visible system behavior aligned with expected outcomes.

This was particularly important in evaluating system usability and error-handling behavior.

### **7.1.7 Acceptance Testing**

Acceptance testing ensured that the system satisfied all documented requirements and end-user expectations. Stakeholders reviewed usability, accuracy, output clarity, and workflow navigation.

Feedback confirmed that the system was intuitive, responsive, and aligned with real cyberbullying detection needs.

Test Result Summary:

All defined test cases passed successfully. No major defects were encountered.

## **7.2 Testing Strategies**

A structured testing strategy was followed throughout the project lifecycle. Testing progressed systematically from component-level validation to full-system verification.

### **7.2.1 Test Strategy and Approach**

Testing was performed both manually and programmatically. Detailed execution logs and dataset-based test scripts were used to validate consistency of classifier behavior.

Primary strategic objectives included:

- verifying correctness of text preprocessing and model inputs ensuring ensemble behavior consistently outperformed single classifiers
- verifying error-free interactions between user interface and backend services
- validating prediction reliability under noisy, sarcastic, and ambiguous text

Field testing simulated real-world user activity, while controlled test cases verified logical correctness.

### **7.2.2 Test Objectives**

The following objectives guided all testing activities:

- all fields and forms must operate correctly
- screens and interactions should respond without delay
- invalid or malformed inputs must be handled safely
- predictions should follow expected patterns in comparable research literature
- transitions between system pages must be correct and intuitive

### **7.2.3 Features Tested**

The major system features examined included:

- data entry validation and prevention of duplicate accounts
- correct routing of each navigation link
- prediction accuracy across cyberbullying categories
- correct Soft Voting operation between BoostDT and BagRF
- adherence to expected model training and evaluation workflows

### **7.2.4 Integration Testing Strategy**

Integration testing emphasized early detection of dependency conflicts and data mismatches.

Testing confirmed correct:

- alignment of TF-IDF features with both classifiers
- synchronization of probability outputs into Soft Voting
- interface communication flow with Django APIs

This strategy prevented error propagation into later development stages.

### **7.2.5 Acceptance Criteria**

A prediction system instance was accepted only when it satisfied these conditions:

- accurate execution of classification pipeline
- stable runtime performance
- error-free navigation and submission
- meaningful categories shown without misinterpretation
- compliance with defined requirements

### **7.2.6 Overall Test Results**

All planned test cases executed successfully. The system demonstrated stable performance, logical correctness, and consistent cyberbullying prediction accuracy. The Soft Voting ensemble consistently produced more reliable outcomes compared to evaluating either single classifier independently.

### **7.2.7 Conclusion**

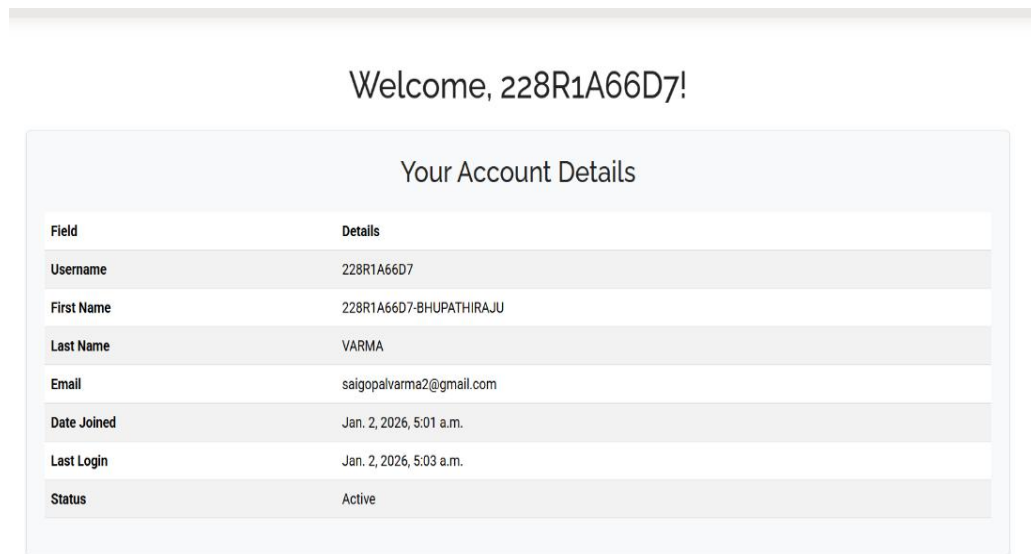
System testing confirmed that the developed ensemble-based cyberbullying detection system satisfies functional expectations, operates reliably under diverse input conditions, and integrates all modules effectively. Through rigorous testing strategies, the project achieved robustness, user readiness, and high classification dependability.

### 7.3 Sample Test Cases

| S No. | Test Case               | Expected Result                                                             | Result | Remarks (if any)                                             |
|-------|-------------------------|-----------------------------------------------------------------------------|--------|--------------------------------------------------------------|
| 01.   | User Login              | User is authenticated and granted access to their dashboard                 | Pass   | Verify incorrect credentials show appropriate error messages |
| 02.   | User Registration       | New user account is created and stored in the database                      | Pass   | Validate email format and duplicate-account handling         |
| 03.   | User Account Activation | Registered user becomes active and can log in successfully                  | Pass   | Ensure inactive users cannot access the system               |
| 04.   | Text-Based Prediction   | System processes submitted text and displays correct cyberbullying category | Pass   | Test normal, abusive, sarcasm- hidden, and empty inputs      |
| 05.   | Voice-Based Prediction  | Voice input is converted to text and classified accurately                  | Pass   | Check microphone permissions and noisy-audio handling        |
| 06.   | Admin Panel             | Admin can view users, predictions                                           | Pass   | Ensure only authorized admins can access this module         |

**Table no 7.3** Test Cases

### Test Case 1:



**Fig 7.3.1** User Login

**Description:** The user enters valid login credentials and submits the form. The system authenticates the credentials and redirects the user to the dashboard, displaying a personalized welcome message and complete account details such as username, email, status, and last login time. Successful login confirms that authentication and user-profile retrieval are working correctly.

### Test Case 2:



**Fig 7.3.2** User Registration

**Description:** After submitting valid registration details, the system successfully creates the user account and displays a confirmation message indicating that registration is complete. The message also informs the user that their account will remain inactive until approved by the administrator. This confirms that the registration workflow and approval control mechanism are functioning correctly.

### Test Case 3:

|   |            |                         |                         |       |                         |          |            |
|---|------------|-------------------------|-------------------------|-------|-------------------------|----------|------------|
| 9 | 228R1A66D5 | saigopalvarma@gmail.com | 228R1A66D7-BHUPATHIRAJU | VARMA | Jan. 2, 2026, 5:17 a.m. | Active   | Deactivate |
| 9 | 228R1A66D5 | saigopalvarma@gmail.com | 228R1A66D7-BHUPATHIRAJU | VARMA | Jan. 2, 2026, 5:17 a.m. | Inactive | Activate   |

User 228R1A66D5 has been activated.

**Fig.7.3.3** User Account Activation

**Description:** When the administrator approves the newly registered user, the system changes the account status from Inactive to Active and displays a confirmation message indicating successful activation. The user is now allowed to log in and access the system. This verifies that the admin-controlled activation process works correctly and prevents unauthorized access before approval.

### Test Case 4:

### Text Analysis

Enter text:

That outfit is awful, idiot do you even own a mirror?" "This is why you'll never be successful

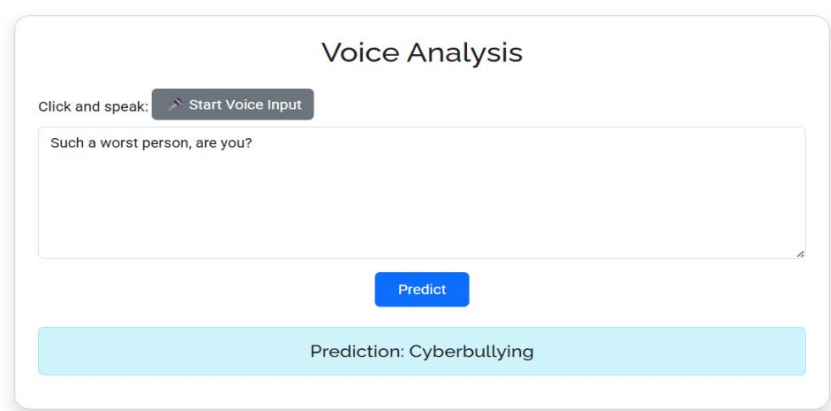
Predict

Prediction: Cyberbullying

**Fig. 7.3.4** Text-Based Prediction

**Description:** The user enters a piece of text into the input box and submits it for analysis. The system processes the text through the preprocessing pipeline and ensemble classifier, then displays the predicted result as “Cyberbullying.” This confirms that the text-analysis workflow, model integration, and result display components are functioning correctly.

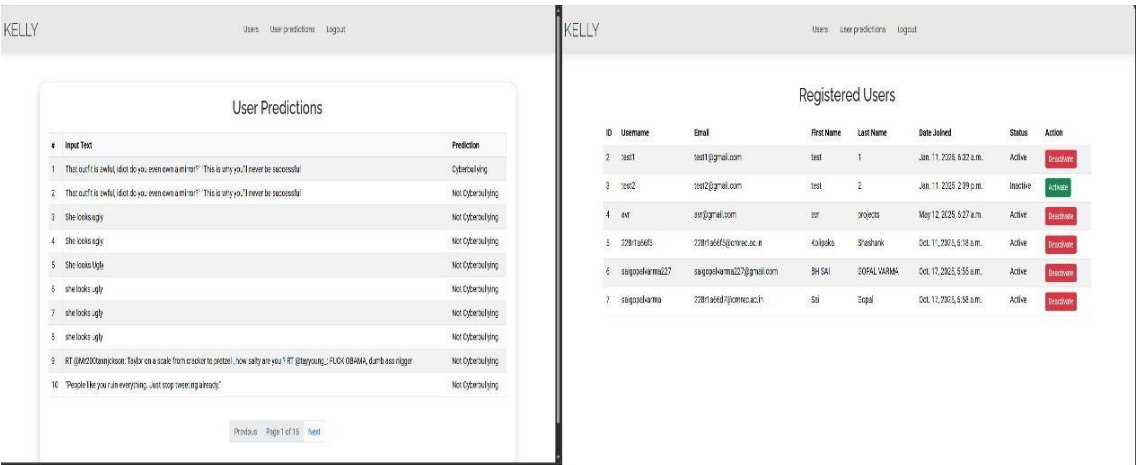
**Test Case 5:**



**Fig. 7.3.5** Voice-Based Prediction

**Description:** The user activates the microphone and speaks a sentence, which is automatically converted to text and displayed in the input area. The system processes the transcribed text using the ensemble classifier and returns the predicted result as “Cyberbullying.” This confirms that the speech- to-text integration, prediction pipeline, and result display are working together correctly.

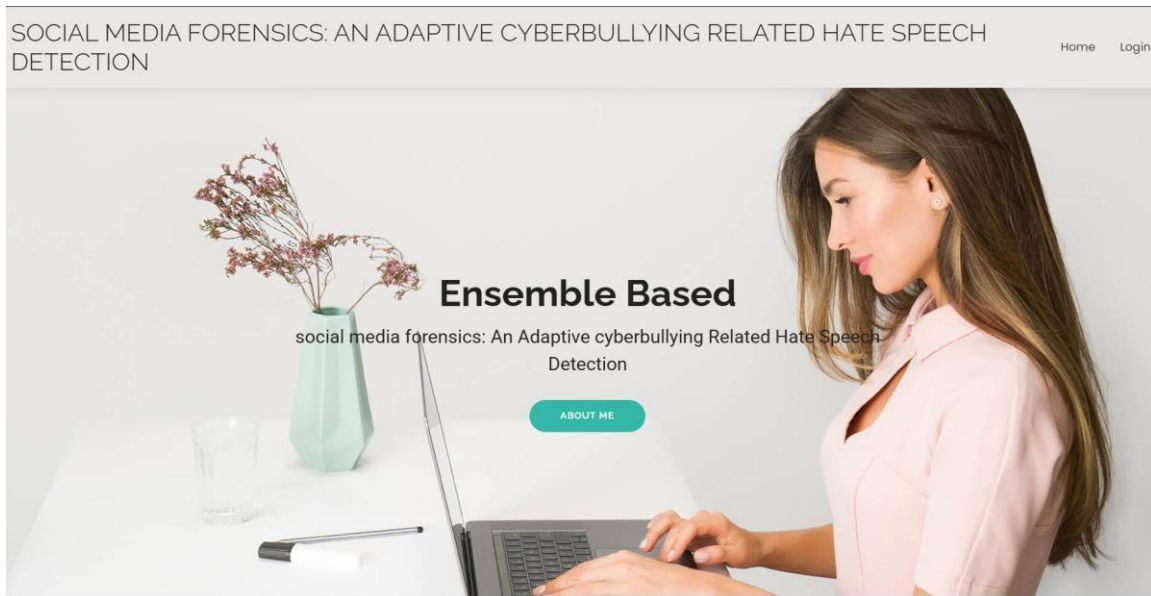
**Test Case 6:**



**Fig. 7.3.6** Admin Panel

**Description:** The administrator logs into the system and navigates to the Admin Panel, where they can view both registered users and user prediction history. The system correctly displays user details, account status, and prediction logs, and allows the admin to activate or deactivate user accounts when required. This confirms that administrative monitoring and access-control functions are working correctly.

## 8. RESULTS

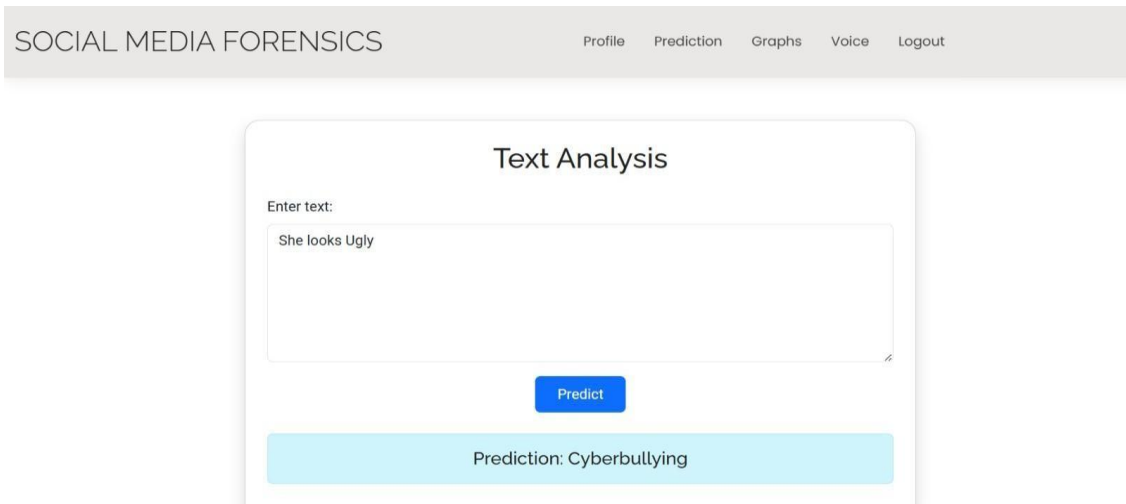


**Fig 8.1** Social Media Forensics Landing Page

**Description:** The primary interface of the application, titled "Cyberbullying Related Hate Speech Detection" system. It introduces the project as a forensic tool powered by advanced Ensemble Machine Learning methods for identifying offensive digital content.

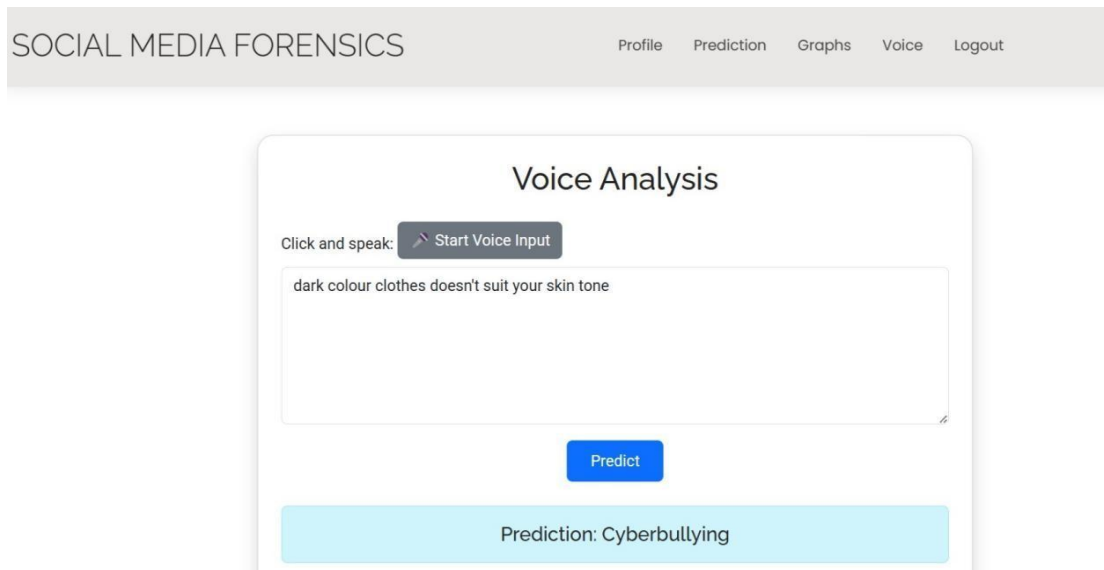
**Fig 8.2** User Authentication & System Access

**Description:** The entry point for the forensics platform, featuring Registration and Login modules. This interface ensures secure access to the ensemble-based detection tools for social media monitoring.



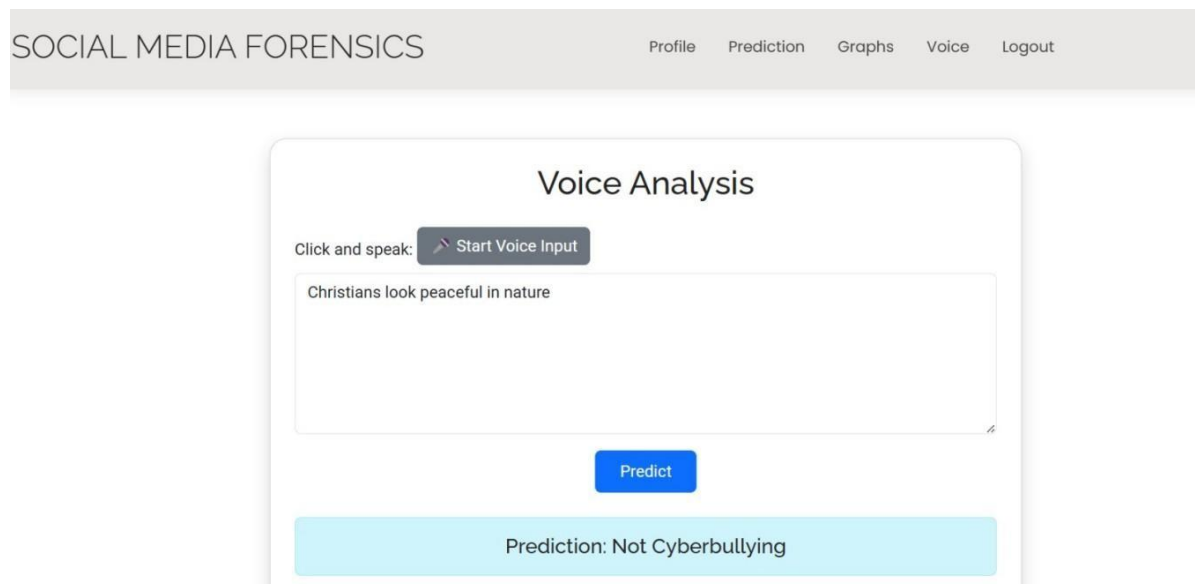
**Fig 8.3** Ensemble-Based Text Classification

**Description:** This screen shows the Text Analysis module in action. The input text is processed by a Soft-Voting Ensemble, which aggregates the probability scores from both the Boosted Decision Tree and Bagging Random Forest to accurately predict "Cyberbullying."



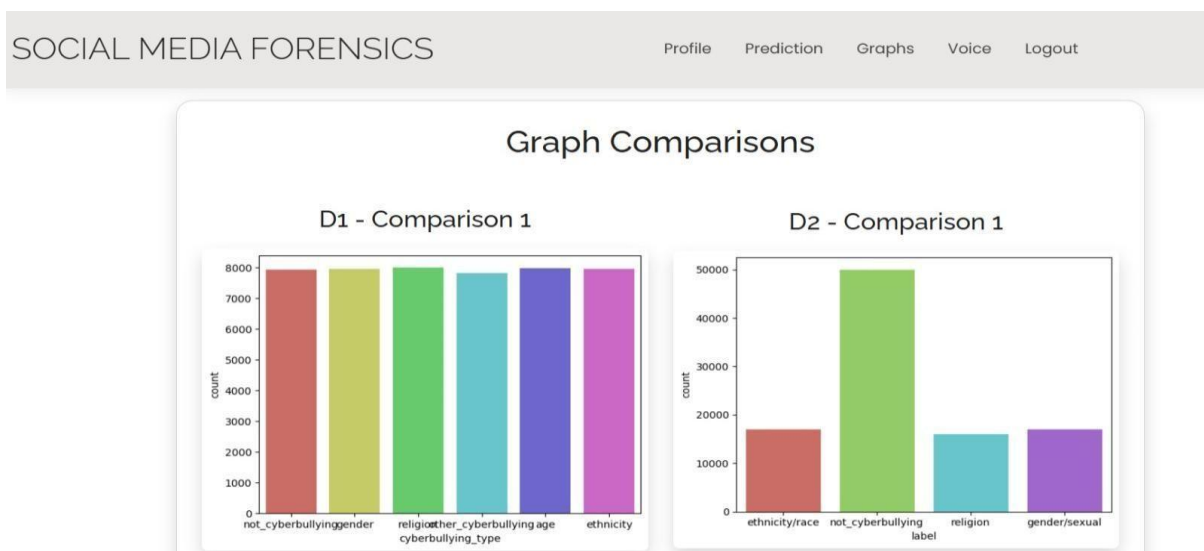
**Fig 8.4** Detection of Subtle Harassment via Voice Input

**Description:** This interface shows the system flagging a discriminatory remark. By combining Boosting and Bagging techniques, the model effectively identifies nuanced offensive speech that single-model architectures might miss.



**Fig 8.5** Voice Transcription & Forensic Analysis

**Description:** This module utilizes speech-to-text conversion to analyze verbal harassment. After transcription, the integrated ensemble model evaluates the text, identifying "Not Cyberbullying" for neutral statements with high precision.



**Fig 8.6** Statistical Data - Class Distribution

**Description:** A dashboard displaying Graph Comparisons across two different datasets. It represents the count of each class across two different datasets.

## 9. CONCLUSION

The present project introduces a comprehensive and intelligent cyberbullying detection framework that effectively leverages ensemble-based machine learning techniques to enhance the accuracy, robustness, and reliability of harmful content classification on social media platforms. The system was implemented using a structured pipeline that includes data collection, preprocessing, feature extraction, imbalance handling, model training, and evaluation, ensuring a systematic and end-to-end approach to solving the problem.

During the implementation phase, raw social media text data was preprocessed through multiple stages, including text cleaning, tokenization, stop-word removal, and normalization, to eliminate noise and improve data quality. The processed text was then transformed into numerical features using the TF-IDF (Term Frequency–Inverse Document Frequency) technique, enabling the machine learning models to capture the importance of words in different contexts. This feature engineering step significantly improved the effectiveness of the classification process.

To address the issue of class imbalance, which is common in cyberbullying datasets, the Synthetic Minority Over-sampling Technique (SMOTE) was applied. SMOTE generates synthetic samples for minority classes, thereby balancing the dataset and preventing the model from being biased toward the majority class. This step enhanced the model's ability to correctly identify minority class instances, particularly subtle or less frequent forms of abusive content.

The core of the system is the ensemble-based classification model, which combines Boosted Decision Tree and Bagging Random Forest algorithms using a soft-voting mechanism. This approach aggregates predictions from multiple models, reducing individual model bias and variance while improving overall generalization. The ensemble model demonstrates strong performance in handling noisy, ambiguous, and linguistically diverse social media text.

The framework supports both binary classification (bullying vs. non-bullying) and multi-class classification (different categories of abusive content), making it adaptable to a wide range of applications. The system was evaluated using standard performance metrics, and the results indicate improved accuracy, stability, and robustness compared to individual classifiers.

The experimental workflow confirms that ensemble learning improves prediction stability compared with individual classifiers, particularly in scenarios involving overlapping or subtle abusive expressions [12]. This validates the effectiveness of the proposed hybrid approach and highlights the advantage of combining multiple models for complex natural language processing tasks.

From a system design perspective, the project followed a structured software engineering methodology, including requirement analysis, feasibility study, system architecture design, UML modeling, and workflow visualization. These steps ensured the development of a modular, scalable, and maintainable system architecture. The modular design supports easy integration of additional components and facilitates future enhancements.

Beyond technical contributions, the project addresses a significant societal issue by promoting safer digital communication environments. Cyberbullying has become a major concern in online platforms, affecting individuals' mental health and well-being. The proposed system provides an automated and scalable solution to detect and mitigate harmful interactions. Additionally, the use of open-source tools and standard computational resources ensures cost-effectiveness and accessibility for academic and organizational use.

In conclusion, the project successfully demonstrates that a well-engineered ensemble-based machine learning framework, combined with effective preprocessing, TF-IDF feature extraction, and SMOTE-based imbalance handling, can significantly improve the performance of automated cyberbullying detection systems. The developed system not only achieves its intended objectives but also establishes a strong foundation for future research and real-world deployment.

## 10. FUTURE ENHANCEMENTS

Although the developed framework provides a strong and effective foundation for cyberbullying detection, several enhancements can be implemented to further improve its performance, scalability, adaptability, and real-world applicability. As cyberbullying continues to evolve in complexity across different platforms, continuous improvements in both modeling techniques and system architecture are essential to maintain high detection accuracy and robustness.

One of the primary directions for future enhancement is the integration of advanced deep learning models such as BERT (Bidirectional Encoder Representations from Transformers), RoBERTa, and LSTM-based architectures. These models are capable of capturing deeper contextual and semantic relationships within textual data, enabling more accurate identification of subtle, implicit, and context-dependent abusive expressions. Transformer-based architectures, in particular, can significantly improve the system's ability to understand sarcasm, slang, and evolving language patterns commonly observed in social media communication. A hybrid approach that combines these deep learning models with the existing ensemble framework can further enhance prediction performance and robustness.

Another important enhancement involves expanding the dataset to include multilingual and cross-domain content. Most current implementations are limited to English-language datasets, which restricts usability in diverse linguistic environments. Extending the system to support multiple languages using multilingual embeddings or transformer models would increase its applicability across different regions and user groups. Additionally, incorporating multimodal data such as images, videos, audio, and combined text-media inputs would allow the system to detect cyberbullying in more complex scenarios, such as memes and visual content, which are increasingly prevalent in online platforms.

The system can also be extended to support real-time monitoring and streaming capabilities. By integrating with live data sources such as social media APIs or message streams, the framework can continuously analyze incoming content and provide instant detection of harmful interactions. This would enable proactive moderation and immediate intervention, making the system suitable for deployment in real-world, large-scale environments such as social media platforms, online forums, and educational systems.

Furthermore, the existing Django-based RESTful backend can be enhanced to improve scalability and performance for production-level deployment. This includes optimizing API endpoints, implementing caching mechanisms, enabling pagination for large datasets, and introducing rate limiting to handle high volumes of requests efficiently. The incorporation of asynchronous processing techniques, such as background task queues, can further improve system responsiveness and reduce latency in real-time detection scenarios.

In addition, introducing adaptive learning mechanisms can significantly improve the system over time. By incorporating feedback loops where users or moderators validate predictions, the system can iteratively retrain and refine its models. This human-in-the-loop approach helps reduce misclassifications, improves generalization, and ensures that the system adapts to evolving patterns of cyberbullying behavior. Periodic retraining with updated datasets can also maintain long-term effectiveness.

From an analytical and usability perspective, the integration of interactive dashboards and visualization tools can enhance the practical utility of the system. These dashboards can provide insights such as frequency of cyberbullying incidents, category-wise distribution, severity analysis, and temporal trends. Such features would assist administrators, researchers, and policymakers in making informed decisions and developing effective prevention strategies.

Finally, strengthening privacy, security, and ethical considerations is essential for real-world deployment. Future improvements can include implementing data anonymization, secure data storage, and compliance with data protection standards. Incorporating fairness-aware machine learning techniques can help minimize bias in predictions, while explainable AI (XAI) methods can provide transparency into model decisions, thereby increasing user trust and system accountability.

Collectively, these enhancements will transform the current framework into a more intelligent, scalable, and user-centered solution. By integrating advanced modeling techniques, expanding data diversity, enabling real-time capabilities, optimizing backend performance, and ensuring ethical deployment, the system can evolve into a comprehensive platform for effective cyberbullying prevention and safer digital communication.

## REFERENCES

1. J. Immanuel D., N. S. Nivetha, M. S. Sharmitha, T. K. Sharma, and M. Gowthamani, “Real-Time Cyberbullying Detection Using NLP and Ensemble Learning Techniques,” in Proc. 7th Int. Conf. Mobile Computing and Sustainable Informatics (ICMCSI), IEEE, 2026.
2. R. Debnath and P. Banerjee, “Soft-Voting Ensemble Strategies for Toxic Comment Classification,” IEEE Transactions on Computational Social Systems, 2025.
3. A. N. Kumar, S. Patel, and R. Mishra, “Boosting-Based Approaches for Online Abuse and Cyberbullying Detection,” Applied Intelligence, 2025.
4. I. Uddin, F. Rahman, and T. Kabir, “Voting Classifiers for Improving Toxic Speech Prediction Accuracy,” Computers & Electrical Engineering, 2025.
5. H. Allwaibed, A. Alotaibi, and A. Alzahrani, “Cyberbullying Detection Approaches for Arabic Texts Using Ensemble Models,” Frontiers in Artificial Intelligence, 2025.
6. Edem Suresh Babu and G. S. Sravanthi, “Sarcasm Detection of Feature Augmentation Using Multiobjective Genetic Algorithm,” Evolution in Computational Intelligence, SIST, vol. 436, pp. 457–466, 2025.
7. J. Morales, D. Santos, and P. Castillo, “Ensemble Modeling for Hate Speech Identification Across Platforms,” IEEE Access, 2024.
8. T. Saha and R. Kar, “Random-Forest-Driven Cyberbullying Detection on Microblog Data,” Journal of Information Security and Applications, 2024.
9. L. Chen, Y. Wang, and Z. Zhao, “A Multi-Stage Ensemble Architecture for Detecting Abusive Language,” Information Sciences, 2024.
10. S. Sihab-Us-Sakib, M. Rahman, and M. Hasan, “Cyberbullying Detection of Resource-Constrained Language Using Transformer-Based Ensembles,” Natural Language Processing Journal, 2024.
11. I. Hussain, A. Ullah, and M. N. Khan, “Differential Transfer Learning with Ensemble Optimization for Hate Speech Detection,” Scientific Reports, 2024.
12. P. Shah and A. Shah, “Comparative Evaluation of Voting, Bagging, and Boosting for Hate Speech Detection,” Procedia Computer Science, 2023.

13. M. Jain and S. Gupta, "An Ensemble-Driven Moderation Framework for Social Media Platforms," IEEE International Conference on Data Science and Advanced Analytics, 2023.
14. R. Biswas, M. Paul, and A. Das, "Deep-Hybrid Ensembles for Detecting Cyberbullying Patterns," arXiv Preprint, 2023.
15. S. W. Azumah, P. Oppong, and A. Mensah, "Adversarial-Aware Ensemble Approaches for Hate Speech Detection," arXiv Preprint, 2023.
16. A. Toktarova, S. Khusainova, and M. Fayzullina, "Comparative Study of Ensemble and Single Models for Social Network Hate Speech," International Journal of Advanced Computer Science and Applications, 2023.
17. M. S. Jahan and M. Oussalah, "A Systematic Review of Hate Speech Detection with Ensemble NLP Models," Neurocomputing, 2023.
18. T. Mahmud, R. Hasan, and S. Rahman, "Ensemble-Based Detection for Low-Resource Cyberbullying Datasets," Information Processing & Management, 2023.
19. A. Ahmed and F. Khan, "Machine Learning and Ensemble Techniques for Cyberbullying and Hate Speech," CEUR Workshop Proceedings, 2023.
20. J. Sathya and F. M. H. Fernandez, "Ontology-Assisted Skip-Gram and Ensemble Models for Cyberbullying Detection," International Conference on Communication and Electronics Systems (ICCES), 2023.